

INTRODUCTION TO JAVA

UNIT I

AGENDA

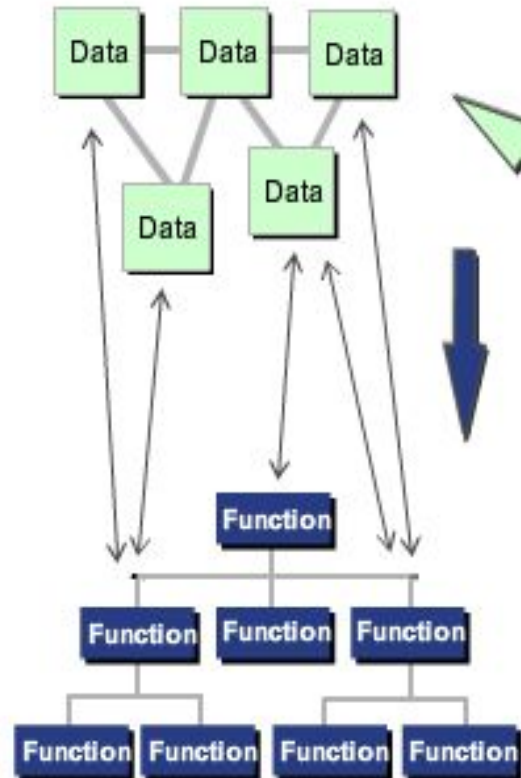
- Introduction to JAVA
- Two Paradigms
- Object Oriented Programming Principles
- Sample Programs

Introduction to java

- ✓ **Object oriented programming** (OOP) is the core of Java programming.
- ✓ Java is a general purpose, object- oriented programming language developed at **Sun Microsystems**. It was invented by James Gosling and his team atrick Naughton, Chris Warth, Ed Frank, and Mike Sheridan in 1991 and was initially called as **Oak**. Renamed as JAVA in 1995.
- ✓ The most important feature that made Java very popular was the “**Platform- Independent**” approach.
- ✓ It was the first programming language that did not tie-up with any particular operating system(or hardware) rather Java programs can be executed anywhere and on any system.
- ✓ Java was designed for the development of the software for consumer electronic devices like TVs, VCRs, etc.

Procedural:

Separation of data and functions

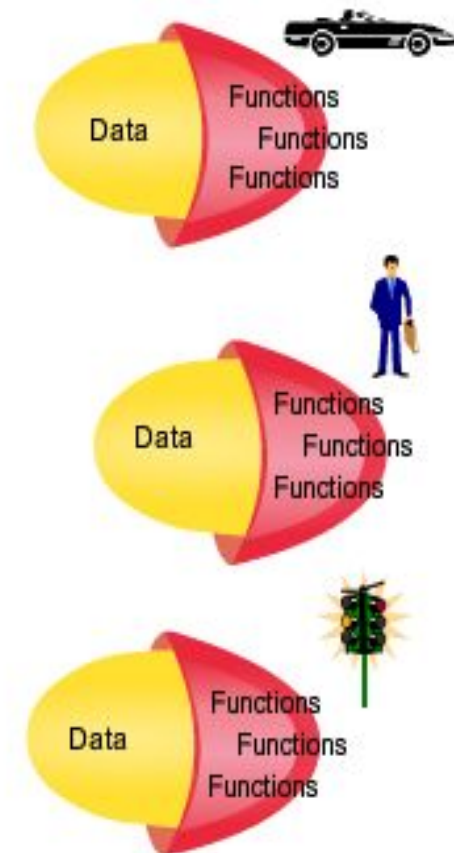


Real world



Object-oriented:

Encapsulation of data and functions



The OOP principles

The basic principles of any object oriented programming language.

- ✓ Encapsulation
- ✓ Inheritance
- ✓ Polymorphism

encapsulation

- ✓ Encapsulation is the mechanism that binds together code and data it manipulates, and keeps both safe from outside interference and misuse.
- ✓ A class is a collection of data and code. An object is an instance of a class. That is, several objects share a common structure (data) and behavior (code) defined by that class.
- ✓ A class is a logical entity (or prototype) and an object is a physical entity.

encapsulation

- ✓ The elements inside the class are known as members.
- ✓ Specifically, the data or variables inside the class are called as member variables or instance variables or data members.
- ✓ The code that operates on these data is referred to as member methods or methods
- ✓ Another purpose of the class is to hide the information from outside manipulation.
- ✓ Class uses public and private interfaces. The members declared as private can only be accessed by the members of that class, whereas, the public members can be accessed from outside the class.

STRUCTURE OF JAVA PROGRAM

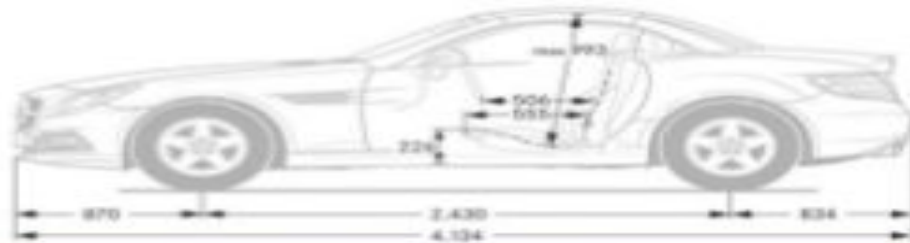


Structure of Java Program

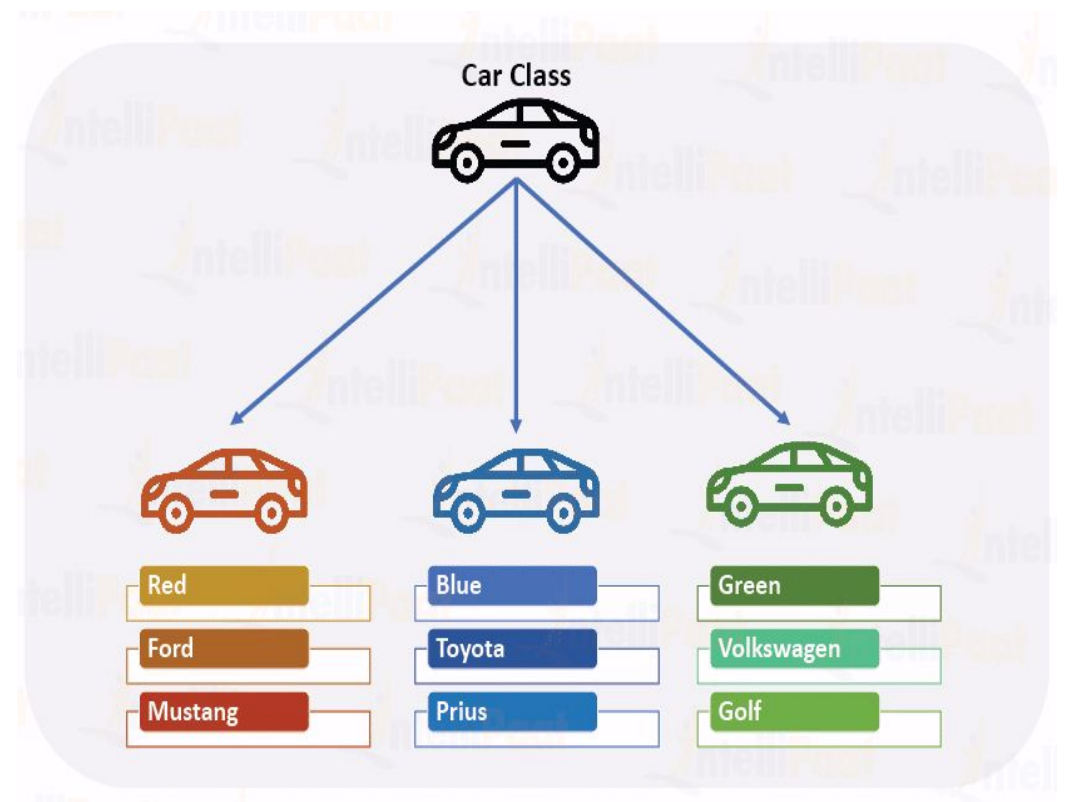
Class fundamentals

- ✓ A Class is a user-defined data type in Java that defines a template or blueprint for an object.
- ✓ We can create any number of variables (objects) of that data type.
- ✓ So, we can say that class is a template for an object and an object is an instance of a class.
- ✓ Most of the times, the terms object and instance are used interchangeably.

Class fundamentals



Think of the class like the Mercedes blue-print and the object like your brand new Mercedes.



Class fundamentals



Classes & Objects in Java

Class

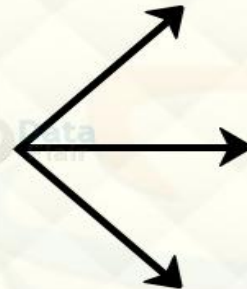
Car

**Data
Members**

model
color
brand

Methods

speed()
size()



model - Ertiga
color - Mehroon
brand - Maruti



model - XUV500
color - Black
brand - Mahindra



model - Swift
color - Red
brand - Maruti

Class fundamentals- declaring class

□ A class contains data (member or instance variables) and the code (member methods) that operate on the data.

□ The general form can be given as –

```
class classname
{
    type var1;
    type var2;
    .....
    type method1(para_list)
    {
        //body of method1
    }
    type method2(para_list)
    {
        //body of method2
    }
    .....
}
```

□ Here, classname is any valid name given to the class.

□ Variables declared within a class are called as instance variables because every instance (or object) of a class contains its own copy of these variables.

□ The code is contained within methods.

□ Methods and instance variables collectively called as members of the class

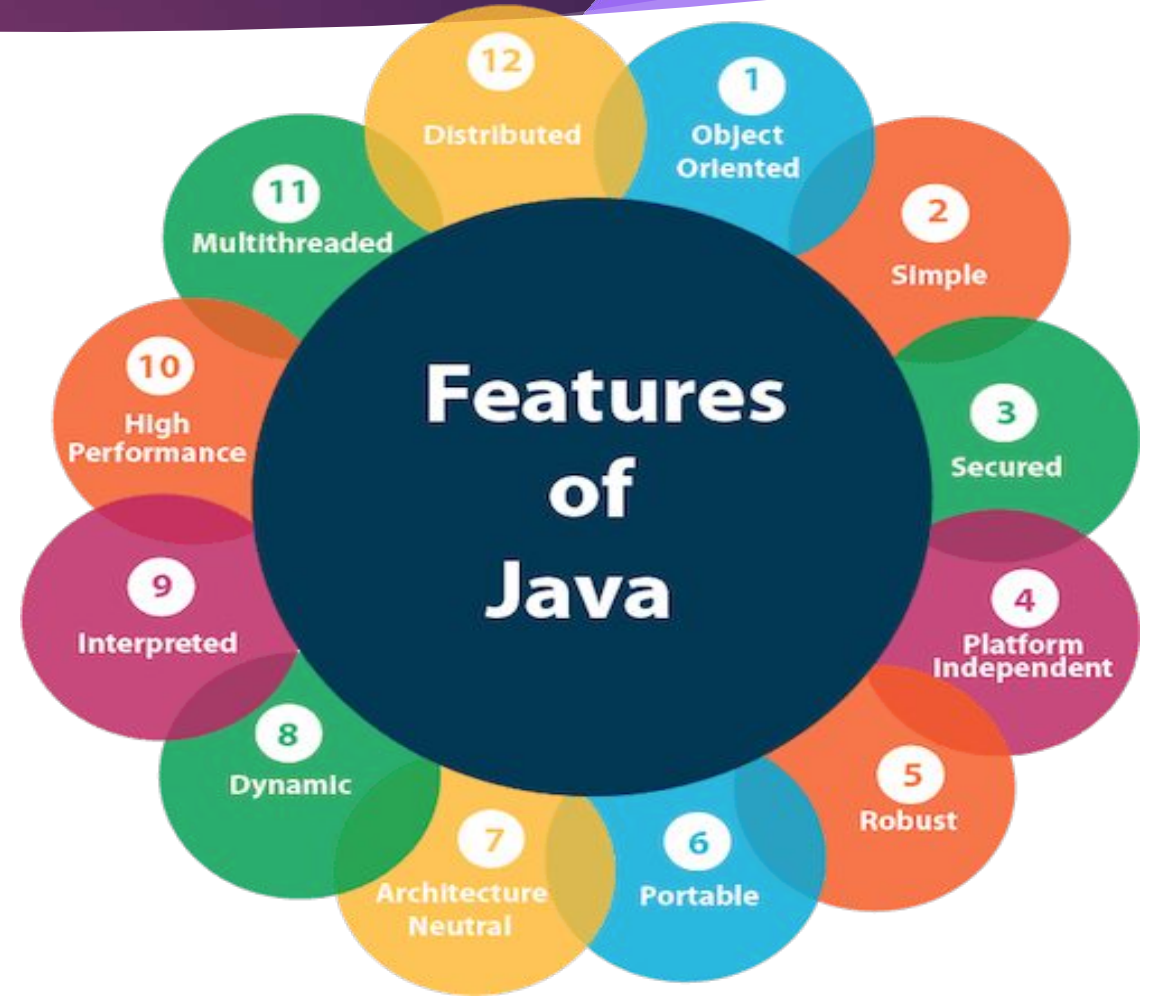
□ The instance variables are acted upon and accessed only the methods defined for that class

Class fundamentals- declaring object

- Creating a class means having a user-defined data type.
- To have a variable of this new data type, we should create an object
- Objects are created dynamically using the new operator
- Syntax: `<classname> <objectname>=new <classname>();`

Features of java/ java buzzwords

Java has become a popular and useful programming language because of its excellent features, which play a very important role in contributing to the popularity of this language. The Java features are called “*Java BuzzWords*”.



Features of java/ java buzzwords

Simple:

- Java is very easy to learn, and its syntax is simple, clean and easy to understand.
- According to Sun Micro Systems, Java language is a simple programming language because:
 - ✓ Java syntax is based on C++ (so easier for programmers to learn it after C++).
 - ✓ Java has removed many complicated and rarely-used features, for example, explicit pointers, operator overloading, etc.
 - ✓ There is no need to remove unreferenced objects because there is an Automatic Garbage Collection in Java.

Features of java/ java buzzwords

Object Oriented:

- Java strongly supports the concepts of **Object-Oriented Programming** due to which it is called a pure object-oriented language.
- Java supports major Object-Oriented programming features like Encapsulation, Abstraction, and Inheritance.
- Almost everything in Java is an object. All programs and data live within objects and classes. ‘Objects’ model Java rather than the ‘processes’. Java comes with an extensible set of classes organized in packages.
- *For example, we cannot develop an executable program in Java without making use of the class.* This indicates that Java very strictly applies the principle of Encapsulation.

Features of java/ java buzzwords

Platform Independent:

- The most significant feature of Java is that it provides **platform independence** which leads to a facility of **portability**, which ultimately becomes its biggest strength.
- Being platform-independent means a program compiled on one machine can be executed on any machine in the world without any change.
- Java achieves platform independence by using the concept of the **BYTE CODE**.
- The Java compiler never converts the source code to machine code like that of the C/C++ compiler.
- Instead, it converts the source code into an intermediate code called the byte code and this byte code is further translated to machine-dependent form by another layer of software called **JVM (Java Virtual Machine)**.

Features of java/ java buzzwords

Platform Independent:

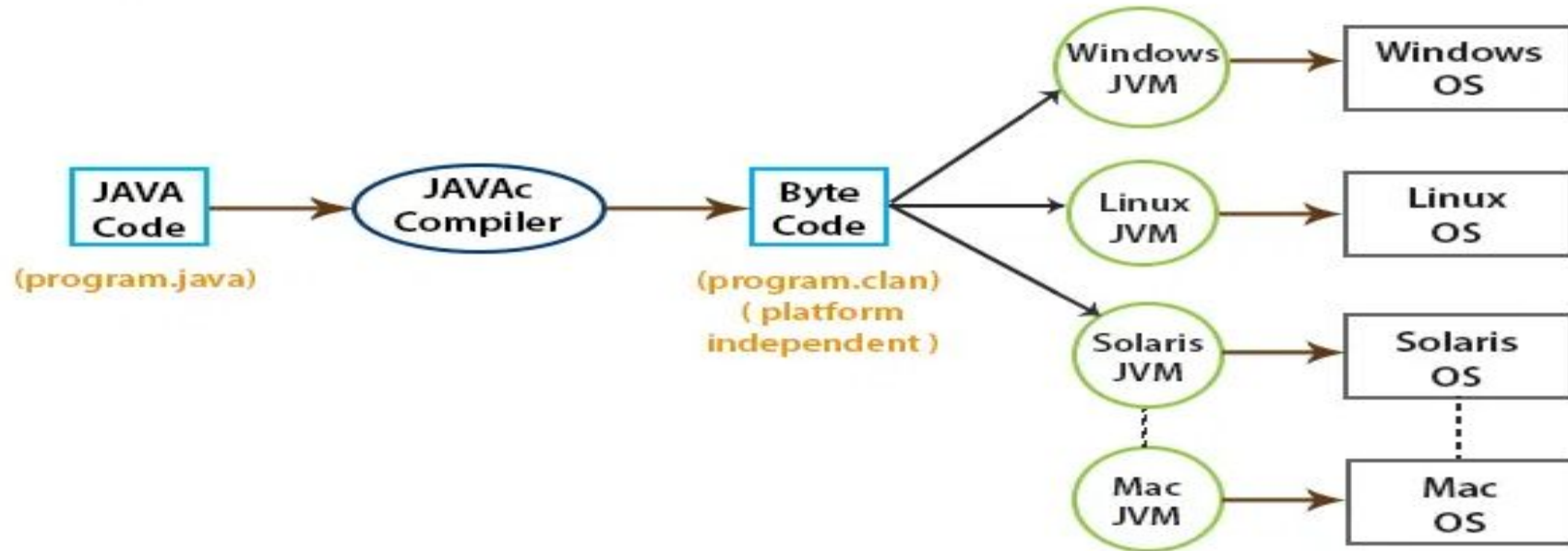
- Therefore, JVM can execute bytecode on any platform or OS on which it is present, regardless of the fact that on which machine the bytecode was generated.
- This is where the **“Write Once, run anywhere” (WORA)** slogan for Java comes in, which means that we can develop applications on one environment (OS) and run on any other environment without doing any modification in the code.
- Below diagram explains the platform independence feature of Java-

Features of java/ java buzzwords

Platform Independent:



Platform Independence in Java



Features of java/ java buzzwords

Platform Independent:

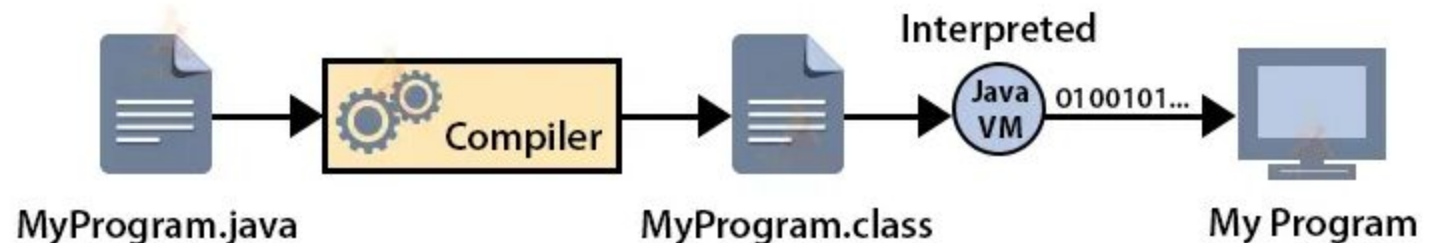
- A platform is the hardware or software environment in which a program runs.
- There are two types of platforms software-based and hardware-based.
- Java provides a software-based platform.
- The Java platform differs from most other platforms in the sense that it is a software-based platform that runs on the top of other hardware-based platforms.
- It has two components:
 - ❖ Runtime Environment
 - ❖ API(Application Programming Interface)

Features of java/ java buzzwords

Interpreted:

- Usually, a computer language can be either compiled or interpreted. Java integrates the power of Compiled Languages with the flexibility of Interpreted Languages.
- **Java compiler (javac)** compiles the java source code into the **bytecode**.
- **Java Virtual Machine (JVM)** then executes this **bytecode** which is executable on many operating systems and is portable.
- The diagram below shows the above process:

Working of Java Virtual Machine

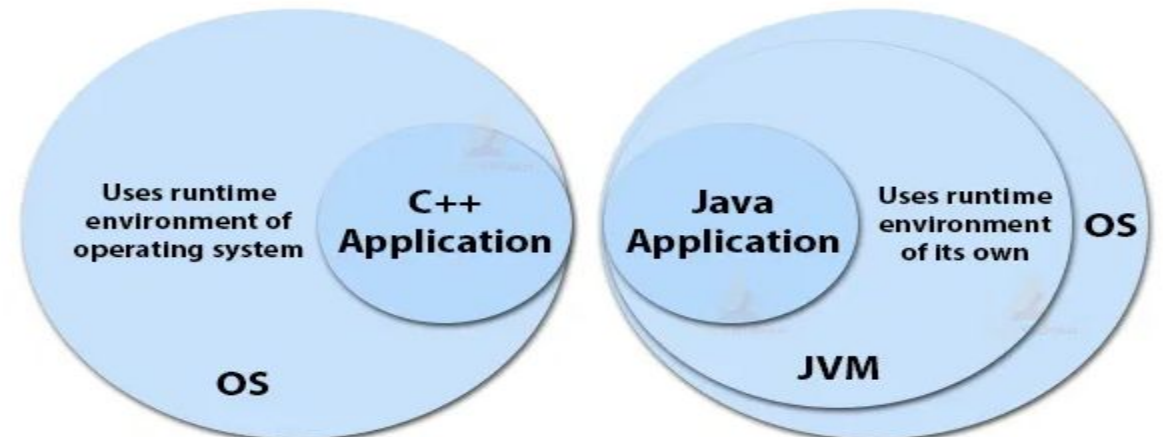


Features of java/ java buzzwords

Secured:

□ Java is best known for its security. With Java, we can develop virus-free systems. Java is secured because:

- ✓ **No explicit pointer**
- ✓ **Java Programs run inside a virtual machine sandbox**-A separate environment that allows users to execute their applications without affecting the underlying system.
- ✓ It has a bytecode verifier that checks the code fragments for any illegal code that violates the access right.



Features of java/ java buzzwords

Portable:

- Java is portable because it facilitates the programmer to carry the Java bytecode to any platform.
- In Java, the size of the primitive data types is machine-independent, which were dependent in the case of C/C++. So, these provisions make Java programs portable among different platforms such as Windows, Unix, Solaris, and Mac.
- Moreover, any changes and updates made in Operating Systems, Processors and System resources will not enforce any changes in Java programs.

Features of java/ java buzzwords

Architecture Neutral:

- This buzzword means that the program written on one platform or OS is independent of other platforms or environments and can run on any other Operating System without recompiling them.
- In other words, it is based on the **‘Write-once-run-anywhere’ (WORA) or ‘Write-once-run-everywhere’ (WORE) approach.**
- Byte-code is not dependent on any machine architecture and Java Virtual Machine (JVM) can easily translate bytecode into a machine-specific code.
- Example, the size of primitive types is fixed- In C programming, int data type occupies 2 bytes of memory for 32-bit architecture and 4 bytes of memory for 64-bit architecture. However, it occupies 4 bytes of memory for both 32 and 64-bit architectures in Java

Features of java/ java buzzwords

Robust:

- Robust simply means strong.
- Java is robust because:

- ✓ It uses **strong memory management**.
- ✓ There is a lack of pointers that avoids security problems.
- ✓ There is **automatic garbage collection** in java which runs on the Java Virtual Machine to get rid of objects which are not being used by a Java application anymore.
- ✓ There are exception handling and the type checking mechanism in Java. All these points make Java robust.

Features of java/ java buzzwords

Dynamic:

- Java is dynamic and extensible means with the help of OOPs, we can add classes and add new methods to classes, creating new classes through subclasses. This makes it easier for us to expand our own classes and even modify them.
- Java gives the facility of dynamically linking new class libraries, methods, and objects. It is highly dynamic as it can adapt to its evolving environment.
- Java even supports functions written in other languages such as C and C++ to be written in Java programs. These functions are called “**native methods**”. These methods are dynamically linked at runtime.

Features of java/ java buzzwords

High Performance:

- The performance of Java is impressive for an interpreted language because of its intermediate bytecode.
- Java provides high performance with the use of **“JIT – Just In Time compiler”**, in which the compiler compiles the code on-demand basis, that is, it compiles only that method which is being called. This saves time and makes it more efficient.
- Java architecture is also designed in such a way that it reduces overheads during runtime. The inclusion of multithreading enhances the overall execution speed of Java programs.
- Bytecodes generated by the Java compiler are highly optimized, so Java Virtual Machine can execute them much faster.

Features of java/ java buzzwords

Multi-Threaded:

- A thread is like a separate program, executing concurrently.
- We can write Java programs that deal with many tasks at once by defining multiple threads.
- The main advantage of multi-threading is that it doesn't occupy memory for each thread. It shares a common memory area.
- Threads are important for multi-media, Web applications, etc.

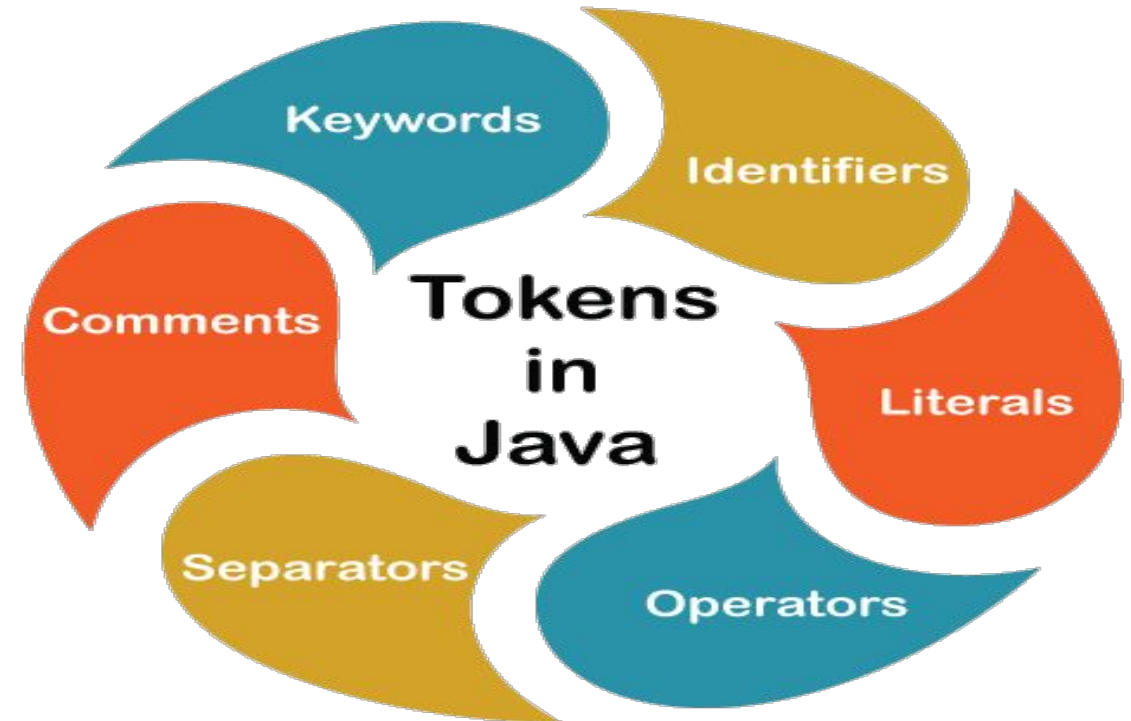
Features of java/ java buzzwords

Distributed:

- Java is distributed because it encourages users to create distributed applications.
- In Java, we can split a program into many parts and store these parts on different computers.
- A Java programmer sitting on a machine can access another program running on the other machine. This feature in Java gives the advantage of distributed programming, which is very helpful when we develop large projects.
- Java helps us to achieve this by providing the concept of **RMI (Remote Method Invocation) and EJB (Enterprise JavaBeans)**.
- Java comes with an extensive library of classes for interacting, using TCP/IP protocols such as HTTP and FTP, which makes creating network connections much easier than in C/C++
- It also enables multiple programmers at many locations to work together on a single project.

Lexical issues

✓ Lets now discuss some atomic elements of JAVA..



COMMENTS

Types of Java Comments

01

Single Line

The single line comment is used to comment only one line.

02

Multi Line

The multi line comment is used to comment multiple lines of code.

03

Documentation

The documentation comment is used to create documentation API. To create documentation API, you need to use javadoc tool.

COMMENTS

Java supports 3 styles of comments

✓ **Multiline comment:** this type of comment begins with `/*` and ends with `*/`

Ex: `/* Welcome toJava Programming */`

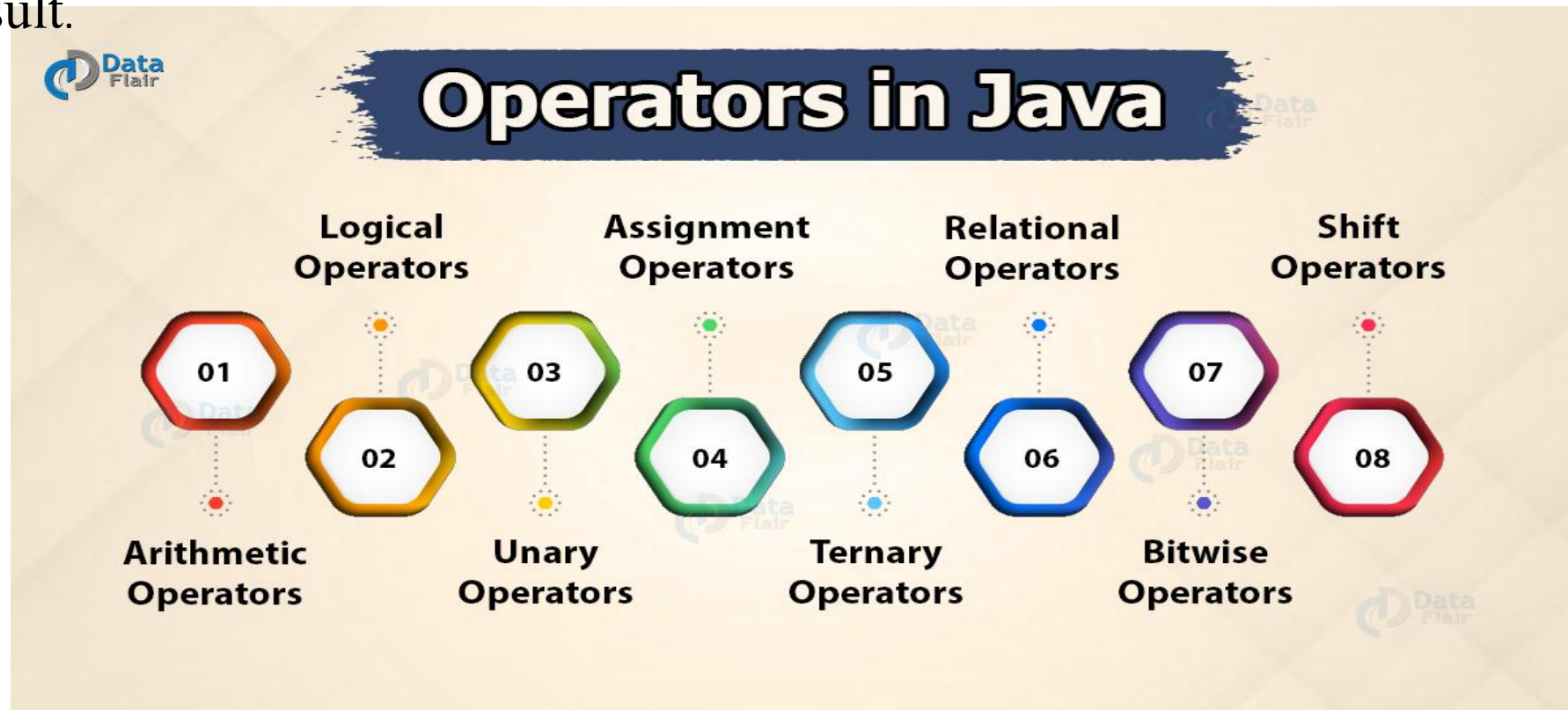
✓ **Single line comments:** this type of comment begins with `//` and ends at the end of current line

Ex: `// Welcome to java Programming`

✓ **Documentation Comment:** This type of comment is used to produce an HTML file that documents your program. The documentation comment begins with `/**` and ends with `*/`

operators

An **operator** is a symbol that takes one or more **arguments (operands)** and operates on them to produce an result.



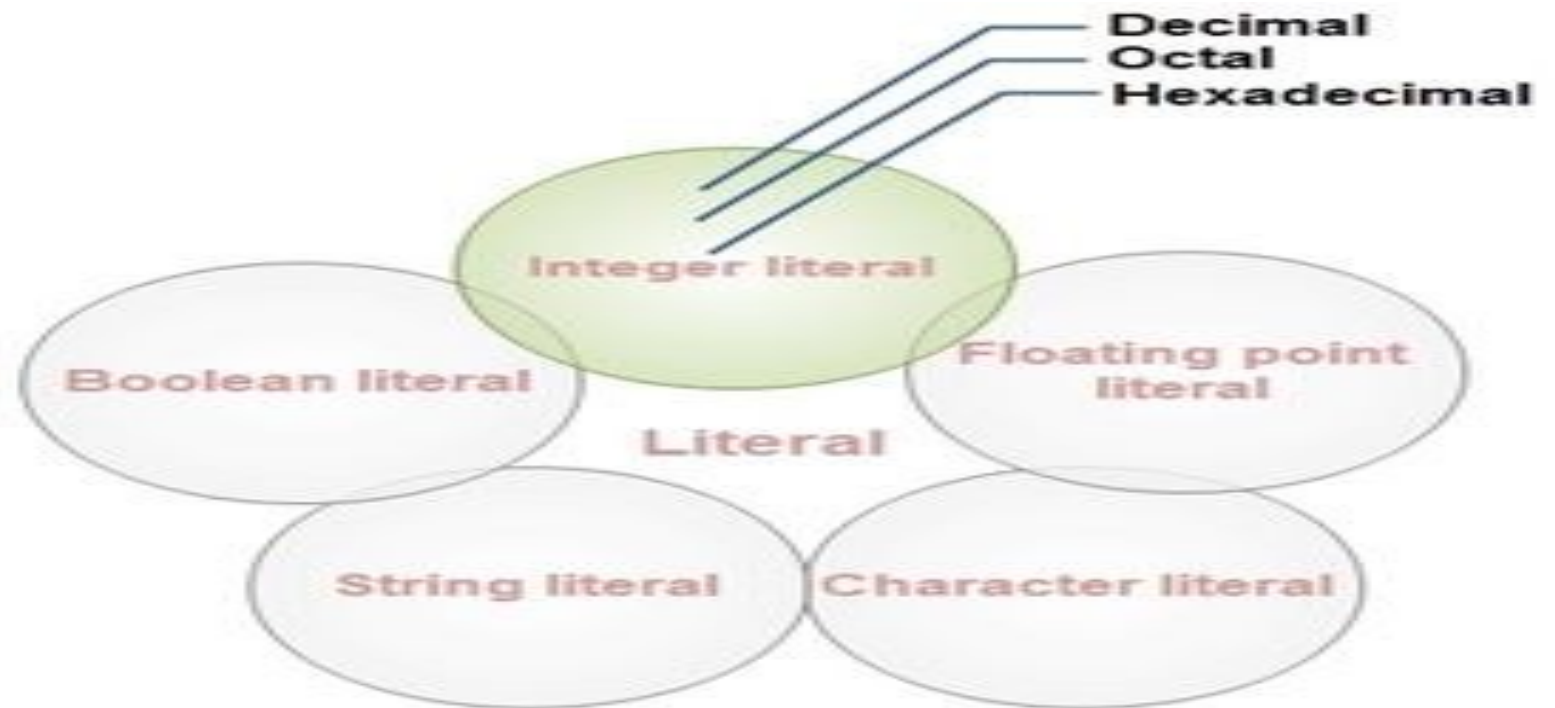
operators

Operator	Category	Precedence
Unary Operator	postfix	expression++ expression--
	prefix	++expression --expression +expression -expression ~!
Arithmetic Operator	multiplication	* / %
	addition	+ -
Shift Operator	shift	<< >> >>>
Relational Operator	comparison	< > <= >= instanceof
	equality	== !=
Bitwise Operator	bitwise AND	&
	bitwise exclusive OR	^
	bitwise inclusive OR	
Logical Operator	logical AND	&&
	logical OR	
Ternary Operator	ternary	? :
Assignment Operator	assignment	= += -= *= /= %= &= ^= = <<= >>= >>>=

literals

□ Literals in Java are sequence of characters that represents constant values to be stored in variables.

□ Five types of Literals:



literals

datatype | keyword

Constant value | Literal

```
int literal = 100;
```

Name of a variable |
identifier

Literals-examples

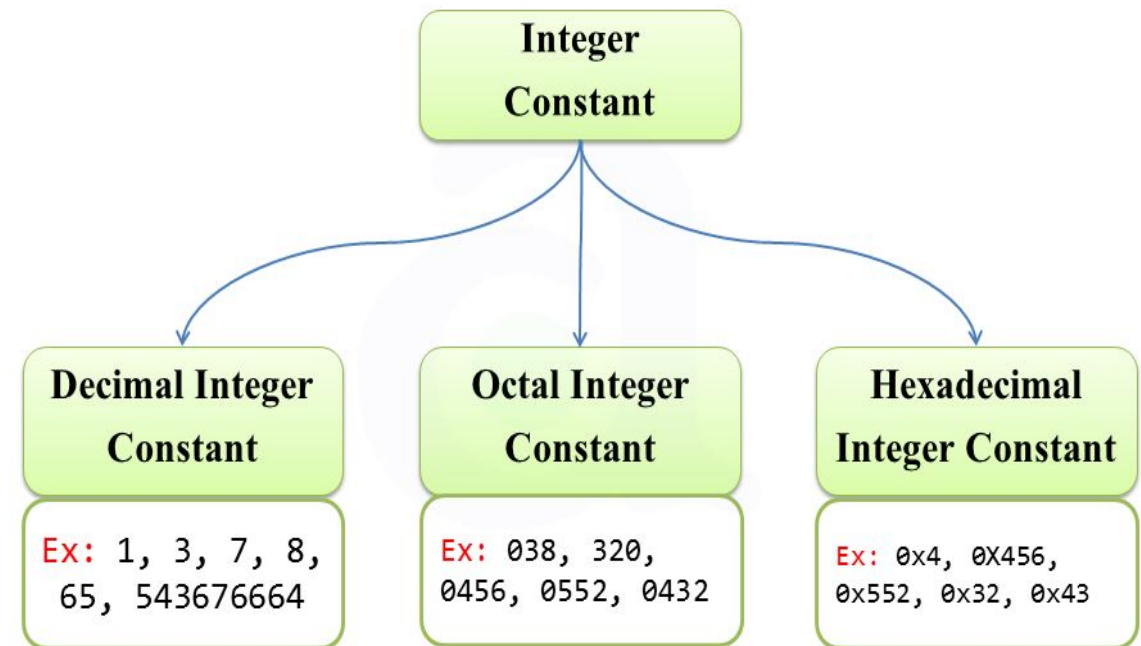
Integer literals -> 10, 5, -8 etc...

Floating-point literals -> 1.2, 0.25, -1.999, etc...

Boolean literals -> true, false

Character literals -> 'a', 'A', 'N', 'q', etc...

String literals -> "hi", "hello", "What's up?"



KEYWORDS

- Java language has 50 words as reserved keywords.
- They implement specific feature of the language.
- The keywords combined with operators and separators according to syntax build the Java language.

KEYWORDS

List of Java Keywords

boolean	byte	char	double	float
short	void	int	long	while
for	do	switch	break	continue
case	default	if	else	try
catch	finally	class	abstract	extends
final	import	new	instance of	private
interface	native	public	package	implements
protected	return	static	super	synchronized
this	throw	throws	transient	volatile

IDENTIFIERS

- In programming languages, identifiers are used for identification purposes.
- In Java, an identifier can be a class name, method name, variable name, object name or label.
- For example :

```
public class Test
{
    public static void main(String[] args)
    {
        int a = 20;
    }
}
```

In the above java code, we have 5 identifiers namely :

- **Test** : class name.
- **main** : method name.
- **String** : predefined class name.
- **args** : variable name.
- **a** : variable name.

Rules for defining Java Identifiers

There are certain rules for defining a valid java identifier. These rules must be followed, otherwise we get compile-time error.

- ✓ The only allowed characters for identifiers are all alphanumeric characters([**A-Z**],[**a-z**],[**0-9**]), '\$'(dollar sign) and '_' (underscore). For example “geek@” is not a valid java identifier as it contain '@' special character.
- ✓ Identifiers should **not** start with digits([**0-9**]). For example “123geeks” is a not a valid java identifier.
- ✓ Java identifiers are **case-sensitive**.
- ✓ There is no limit on the length of the identifier but it is advisable to use an optimum length of 4 – 15 letters only.
- ✓ **Reserved Words** can't be used as an identifier. For example “int while = 20;” is an invalid statement

separators

□ Separators are the symbols that indicates where group of code are divided and arranged.

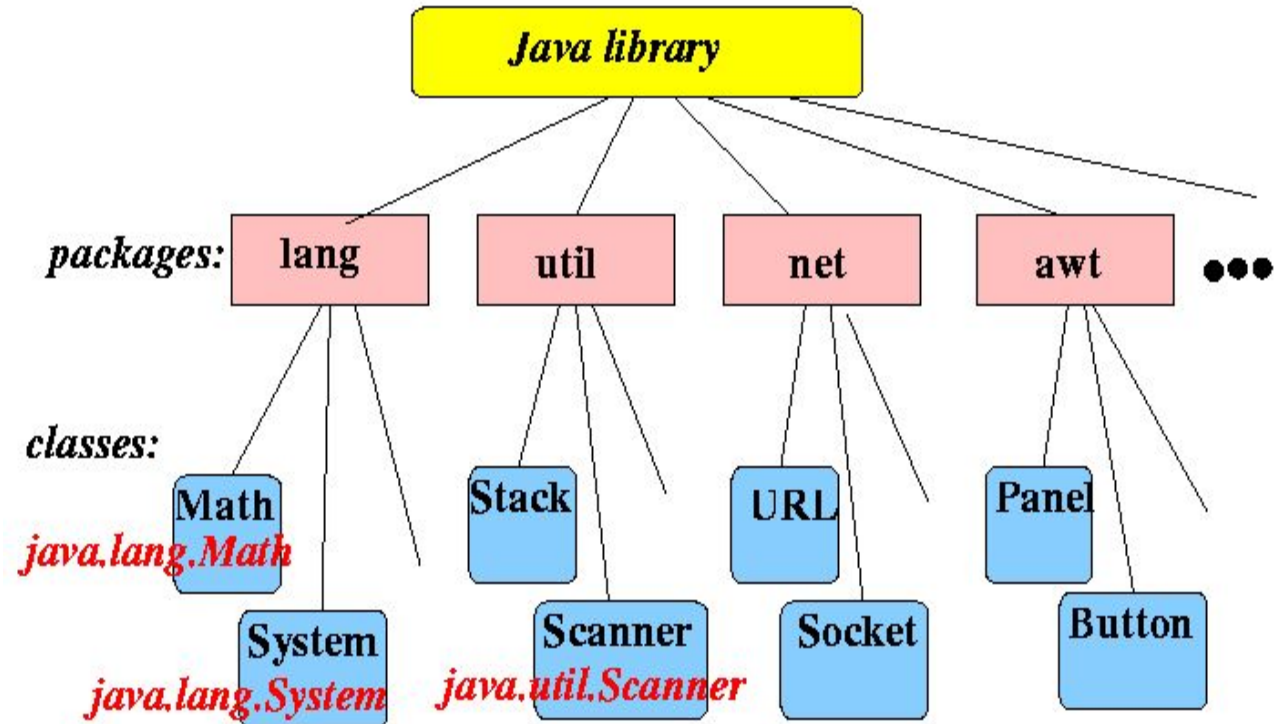
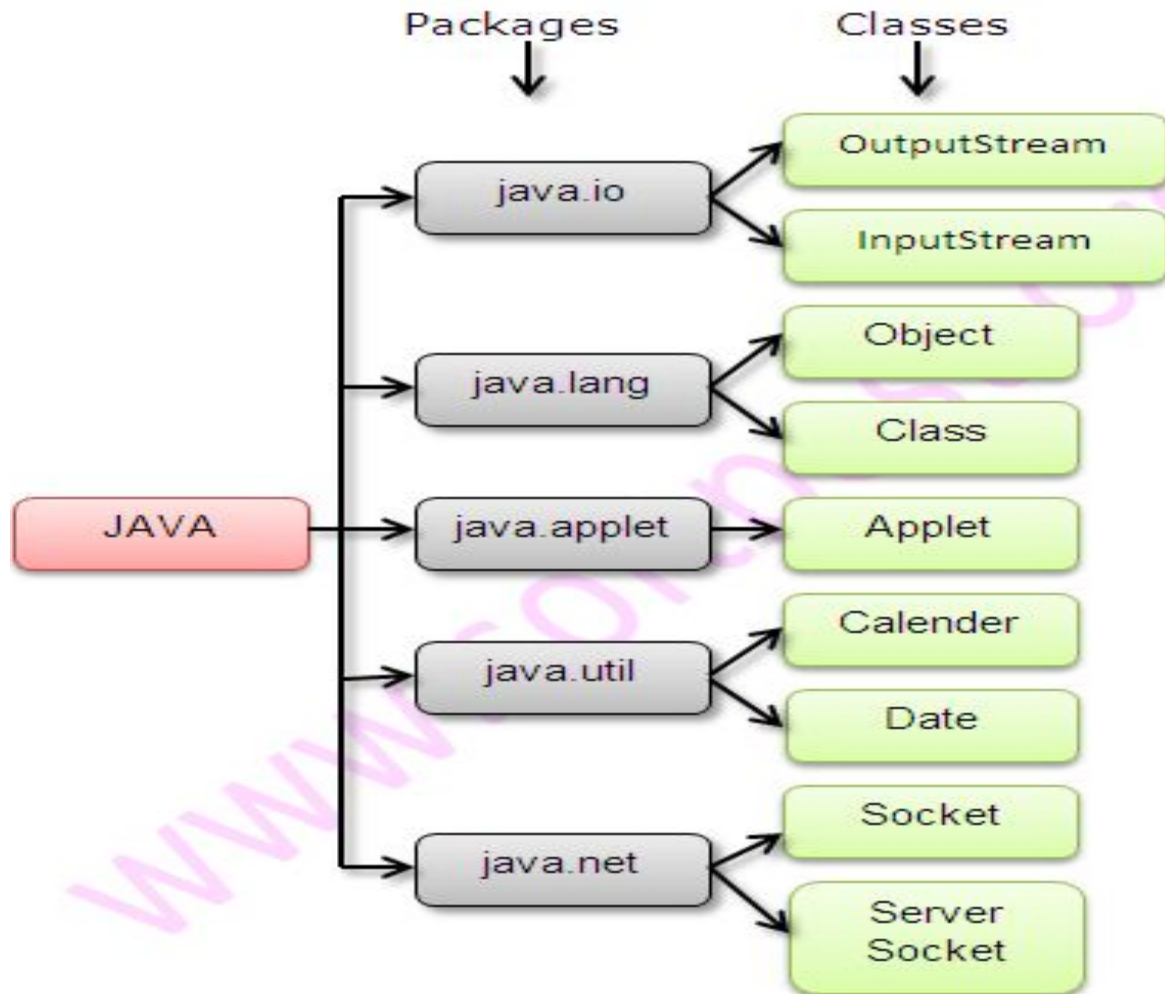
□ Some of the separators are:

Symbol	Name	Purpose
()	Parentheses	Used to contain lists of parameters in method definition and invocation. Also used for defining precedence in expressions, containing expressions in control statements, and surrounding cast types.
{ }	Braces	Used to contain the values of automatically initialized arrays. Also used to define a block of code, for classes, methods, and local scopes.
[]	Brackets	Used to declare array types. Also used when dereferencing array values.
;	Semicolon	Terminates statements.
,	Comma	Separates consecutive identifiers in a variable declaration. Also used to chain statements together inside a for statement.
.	Period	Used to separate package names from subpackages and classes. Also used to separate a variable or method from a reference variable.

Java class libraries

- Java environment has several built in class libraries.
- Java standard library includes hundreds of classes and methods grouped into several functional packages.
- Most commonly used packages are:
 - ✓ Language support Package.
 - ✓ Utilities packages.
 - ✓ Input/output packages
 - ✓ Networking packages
 - ✓ AWT packages.
 - ✓ Applet packages.

Java class libraries



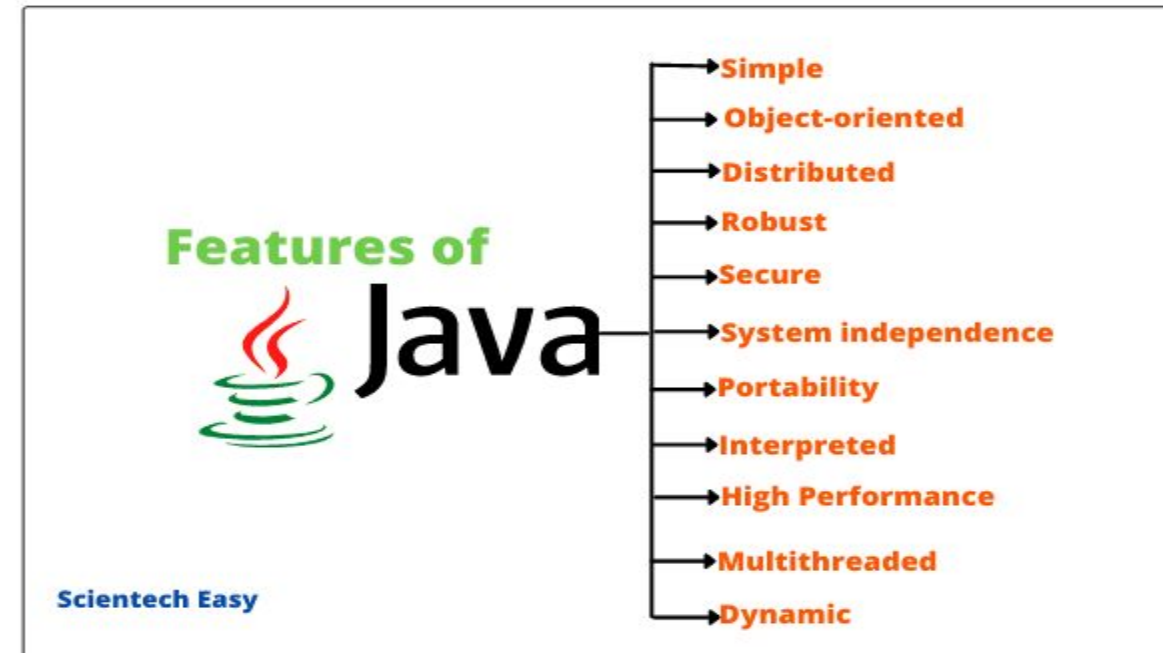
AGENDA

Data Types and Variables

- ✓ Java is a Strongly Typed Language-Justify
- ✓ Primitive Data Types
 - ✓ Integers
 - ✓ Floating Point types
 - ✓ Characters
 - ✓ Boolean

Java is a strongly typed language- Justify

- ✓Java is a strongly typed language.
- ✓Part of Java's safety and robustness comes from this fact



Java is a strongly typed language- Justify

✓Java is a strongly typed language – Justify??

- ❖ Every variable has a type, every expression has a type, and every type is strictly defined
- ❖ All assignments, whether explicit or via parameter passing in method calls are checked for type compatibility
- ✓ Here there are no automation conversions of conflicting types as in some languages
- ✓ The java compiler checks all expressions and parameters to ensure that the types are compatible
- ✓ Any type mismatches are errors that must be corrected before the compiler will finish compilation

Java is a strongly typed language- Justify

✓Example:

The following declaration is allowed because the variable "hasDataType" is declared to be a boolean data type:

```
boolean hasDataType;
```

A variable cannot start off life without knowing the range of values it can hold, and once it is declared, the data type of the variable cannot change.

For the rest of its life, hasDataType can only ever have a value of true or false.

Primitive data types

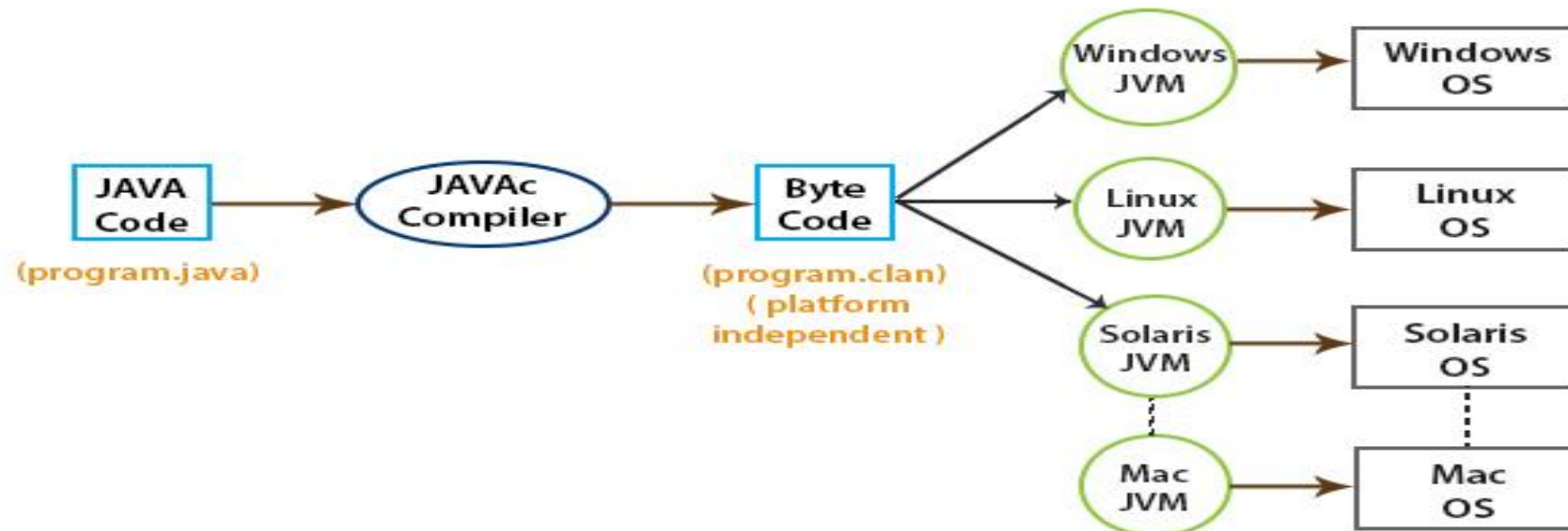
- ✓ Java defines eight primitive types of data: **byte, short, int, long, char, float, double, and boolean**.
- ✓ These can be put in four groups:
 - ✓ **Integers**-This group includes **byte, short, int, and long**, which are for whole-valued signed numbers.
 - ✓ **Floating-point numbers** This group includes **float and double**, which represent numbers with fractional precision.
 - ✓ **Characters** This group includes **char**, which represents symbols in a character set, like letters and numbers.
 - ✓ **Boolean** This group includes **boolean**, which is a special type for representing true/false

Primitive data types

One of the feature of Java is its Portability, which means that java programs can be executed on any operating system/machine

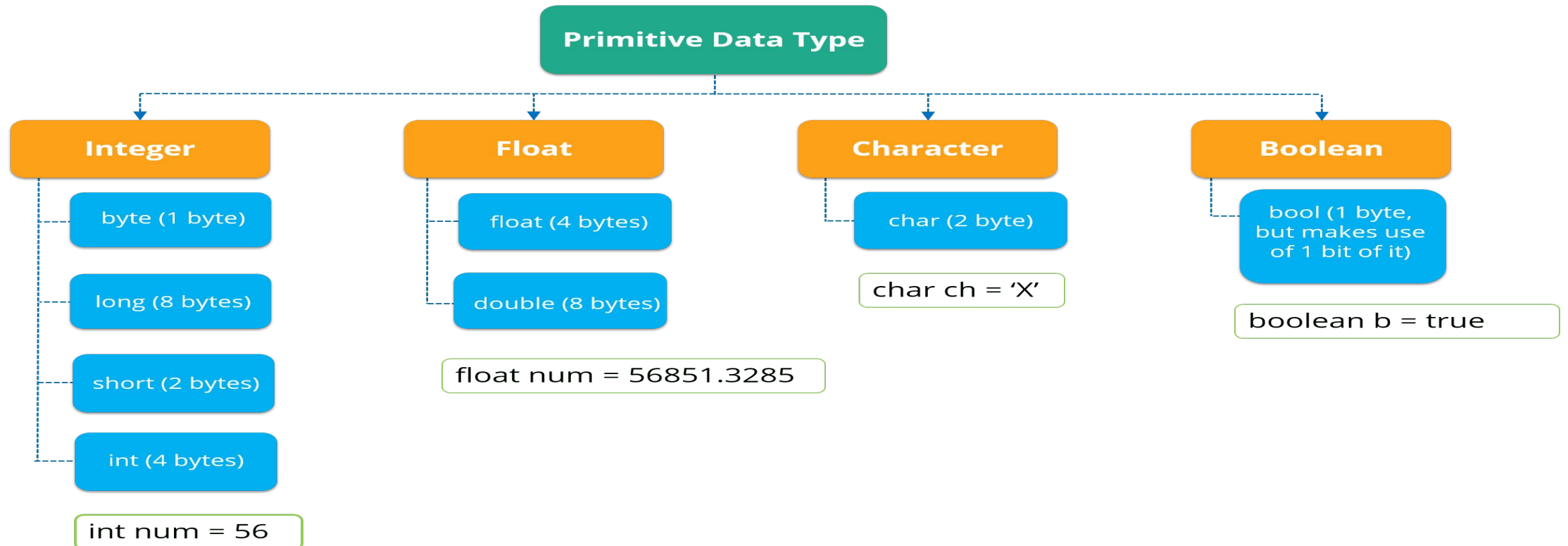


Platform Independence in Java



Primitive data types

Because of Java's Portability, all data types must have a strictly defined size and range of values.



Integers

- ✓ Java defines four integer types: **byte, short, int, and long**
- ✓ **All of these are** whole-valued signed numbers, positive and negative values
- ✓ Java does not support unsigned, positive only integers
- ✓ Examples:
 - byte b, c;
 - short s;
 - short t;
 - int lightspeed;
 - long days;

Integers

✓ The width and range of these integer types are shown below:

Name	Width	Range
long	64	−9,223,372,036,854,775,808 to 9,223,372,036,854,775,807
int	32	−2,147,483,648 to 2,147,483,647
short	16	−32,768 to 32,767
byte	8	−128 to 127

Min Value: $-2^{\text{width}-1}$

Max Value: $2^{\text{width}-1} - 1$

integers

✓ When using a constant bigger than int range, we should suffix it with 'l' or 'L' to indicate it to be a long value.

Eg: long a=131009; is fine

long b=123456789012345; is not fine ✗

long c=123456789012345L; is fine ✓

Integers – program example

Program that computes the number of miles that light will travel in a specified number of days

```
public class Light
{
    public static void main(String args[])
    {
        int lightspeed;
        long days,seconds,distance;
        // approximate speed of light in miles per second
        lightspeed = 186000;
        days = 1000; // specify number of days here
        seconds = days * 24 * 60 * 60; // convert to seconds
        distance = lightspeed * seconds; // compute distance
        System.out.print("In " + days);
        System.out.print(" days light will travel about ");
        System.out.println(distance + " miles.");
    }
}
```

Floating –point types

- ✓ Floating-point numbers, also known as *real* numbers, are used when evaluating expressions that require fractional precision.
- ✓ Examples: calculating of square roots, sine or cosine value, simple interest etc.
- ✓ The type **float** specifies a *single-precision* value that uses 32 bits of storage.
- ✓ Double precision, as denoted by the **double** keyword, uses 64 bits to store a value.
- ✓ When you need to maintain accuracy over many iterative calculations, or are manipulating large-valued numbers, **double** is the best choice.

Floating point types-program example

Program to compute area of circle

```
import java.util.Scanner;
public class Area
{
    public static void main(String[] args)
    {
        double r,area;
        double pi = 3.14;
        Scanner s = new Scanner(System.in);
        System.out.print("Enter radius of circle:");
        r = s.nextDouble();
        area = pi * r * r;
        System.out.println("Area of circle:"+area);
    }
}
```

Boolean s

- ✓ Java has a primitive type, called **boolean**, for logical values.
- ✓ It can have only one of two possible values, **true** or **false**.
- ✓ This is the type returned by all relational operators
- ✓ **boolean** is the type required by the conditional expression that govern the control statements such as **if** and **for**
- ✓ Example: **boolean** flag=true;

characters

- ✓ In Java, the data type used to store characters is char.
- ✓ In C/C++, char is 8 bits wide. This is not the case in Java.
- ✓ Java uses Unicode to represent characters.
- ✓ Unicode defines a fully international character set that can represent all of the characters found in all human languages.
- ✓ It is a unification of dozens of character sets, such as Latin, Greek, Arabic, Cyrillic, Hebrew, Katakana, Hangul, and many more. For this purpose, it requires **16 bits**.
- ✓ The range of a char is **0 to 65,536**

characters-program example

```
// char variables behave like integers.
class CharDemo2 {
    public static void main(String args[]) {
        char ch1;

        ch1 = 'X';
        System.out.println("ch1 contains " + ch1);

        ch1++; // increment ch1
        System.out.println("ch1 is now " + ch1);
    }
}
```

The output generated by this program is shown here:

```
ch1 contains X
ch1 is now Y
```

Boolean -program example

```
class BoolTest {  
    public static void main(String args[]) {  
        boolean b;  
  
        b = false;  
        System.out.println("b is " + b);  
        b = true;  
        System.out.println("b is " + b);  
  
        // a boolean value can control the if statement  
  
        if(b) System.out.println("This is executed.");  
  
        b = false;  
        if(b) System.out.println("This is not executed.");  
  
        // outcome of a relational operator is a boolean value  
        System.out.println("10 > 9 is " + (10 > 9));  
    }  
}
```

The output generated by this program is shown here:

```
b is false  
b is true  
This is executed.  
10 > 9 is true
```

GUESS THE OUTPUT ????

```
char chl = 'x'  
  chl ++;
```

variables

- The variable is the basic unit of storage in a Java program.
- A variable is defined by the combination of an identifier, a type, and an optional initializer.
- In addition, all variables have a scope, which defines their visibility, and a lifetime.
- All variables must be declared before they can be used.

□ Declaring a Variable

type identifier [= *value*][, *identifier* [= *value*] ...] ;

- The *type* is one of Java's atomic types, or the name of a class or interface.
- The *identifier* is the name of the variable.

Variable Initialization

Two ways:

- Using Constants as Initializers

- Initialized Dynamically using any expression at the time when the variable is declared

```
int a, b, c;           // declares three ints, a, b, and c.
int d = 3, e, f = 5;   // declares three more ints, initializing
                       // d and f.
byte z = 22;           // initializes z.
double pi = 3.14159;   // declares an approximation of pi.
char x = 'x';          // the variable x has the value 'x'.
```


Variable Initialization

```
class Example
{
    public static void main(String args[])
    {
        double a=10, b=2.6;
        double c=a/b;
        System.out.println("value of c is"+c);
    }
}
```

The Scope and Lifetime of Variables

- Java allows variables to be declared within any block.
- A block is begun with an opening curly brace and ended by a closing curly brace.
- A block defines a *scope*.
- A scope determines what objects are visible to other parts of your program.
- It also determines the **lifetime of those objects**.
- Many other computer languages define two general categories of scopes: **global** and **local**. However, these traditional scopes do not fit well with Java's strict, object-oriented model.
- As a general rule, variables declared inside a scope are not visible (that is, accessible) to code that is defined outside that scope. Thus, when you declare a variable within a scope, you are localizing that variable and protecting it from unauthorized access and/or modification.

The Scope and Lifetime of Variables

- Variables are created when their scope is entered, and destroyed when their scope is left.
- The lifetime of a variable is confined to its scope.
- Scopes can be nested.
- *The variable at outer scope can be accessed in inner scope but vice versa is not possible.*
- *There should not be two variables with the same name in different scope.*

The Scope and Lifetime of Variables

```
class Scope
{
    public static void main(String args[])
    {
        int x; // known to all code within main
        x = 10;
        if(x == 10) // start new scope
        {
            int y = 20; // known only to this block
            // x and y both known here.
            System.out.println("x and y: " + x + " " + y);
            x = y * 2;
        }
        // y = 100; // Error! y not known here
        // x is still known here.
        System.out.println("x is " + x);
    }
}
```

The Scope and Lifetime of Variables

```
// Demonstrate lifetime of a variable.
class LifeTime {
    public static void main(String args[]) {
        int x;

        for(x = 0; x < 3; x++) {
            int y = -1; // y is initialized each time block is entered
            System.out.println("y is: " + y); // this always prints -1
            y = 100;
            System.out.println("y is now: " + y);
        }
    }
}
```

The output generated by this program is shown here:

```
y is: -1
y is now: 100
y is: -1
y is now: 100
y is: -1
y is now: 100
```

Guess the output --- ???

```
class Scope
{
    public static void main(String[] args)
    {
        int bar =1;
        {
            int bar=2;
        }
    }
}
```

agenda

- Arrays-Introduction

- 1-D Array and Multi-Dimensional Arrays

- ✓ Creation of 2D Array
- ✓ Irregular/Jagged Arrays
- ✓ Initialization

- Example

- Assignment

Introduction to arrays

- An *array* is a group of like-typed variables that are referred to by a common name.
- Arrays of any type can be created and may have one or more dimensions.
- A specific element in an array is accessed by its index.
- Two types:
 - ✓ One-Dimensional Array
 - ✓ Multi- Dimensional Array

Creation of one dimensional arrays

Creating an array is a two step process:

□ **Step 1:** Create an array variable of the desired type

Syntax: `type array_name[];`

Here type is the base type of the array, array_name is an identifier

Example: `int month_days[];`



null

month_days

Creation of one dimensional arrays

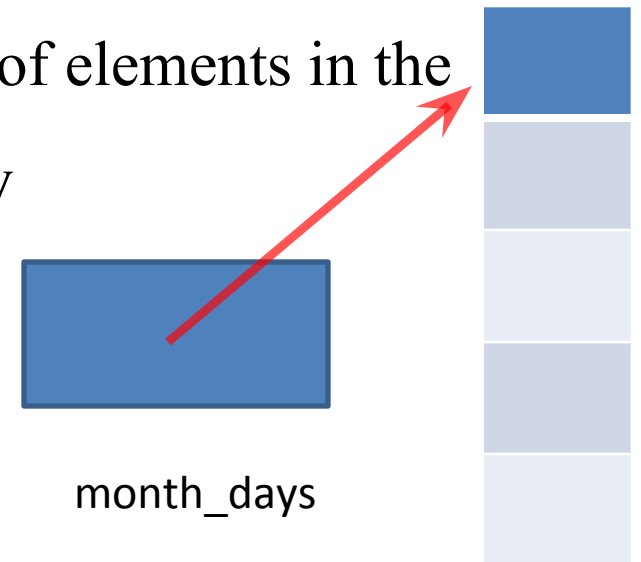
□ **Step 2:** Allocate the memory that will hold the array elements using new and then assign to the array variable.

Syntax: `array_name = new type[size];`

Here type is the base type of the array, size refers to the number of elements in the array and new is the keyword that allocates memory dynamically

Example: `month_days = new int[5];`

Access the third element: `month_days[2]`



example

```
// Demonstrate a one-dimensional array.
class Array {
    public static void main(String args[]) {
        int month_days[];
        month_days = new int[12];
        month_days[0] = 31;
        month_days[1] = 28;
        month_days[2] = 31;
        month_days[3] = 30;
        month_days[4] = 31;
        month_days[5] = 30;
        month_days[6] = 31;

        month_days[7] = 31;
        month_days[8] = 30;
        month_days[9] = 31;
        month_days[10] = 30;
        month_days[11] = 31;
        System.out.println("April has " + month_days[3] + " days.");
    }
}
```

Array initialization

- Arrays can also be initialized when they are declared
- The process is same as that of initializing simple types
- An array initializer is a list of comma separated values surrounded by curly braces
- **Example** : `int month_days[] = { 31,28,30,31,30,31};`

example

```
// An improved version of the previous program.
class AutoArray {
    public static void main(String args[]) {

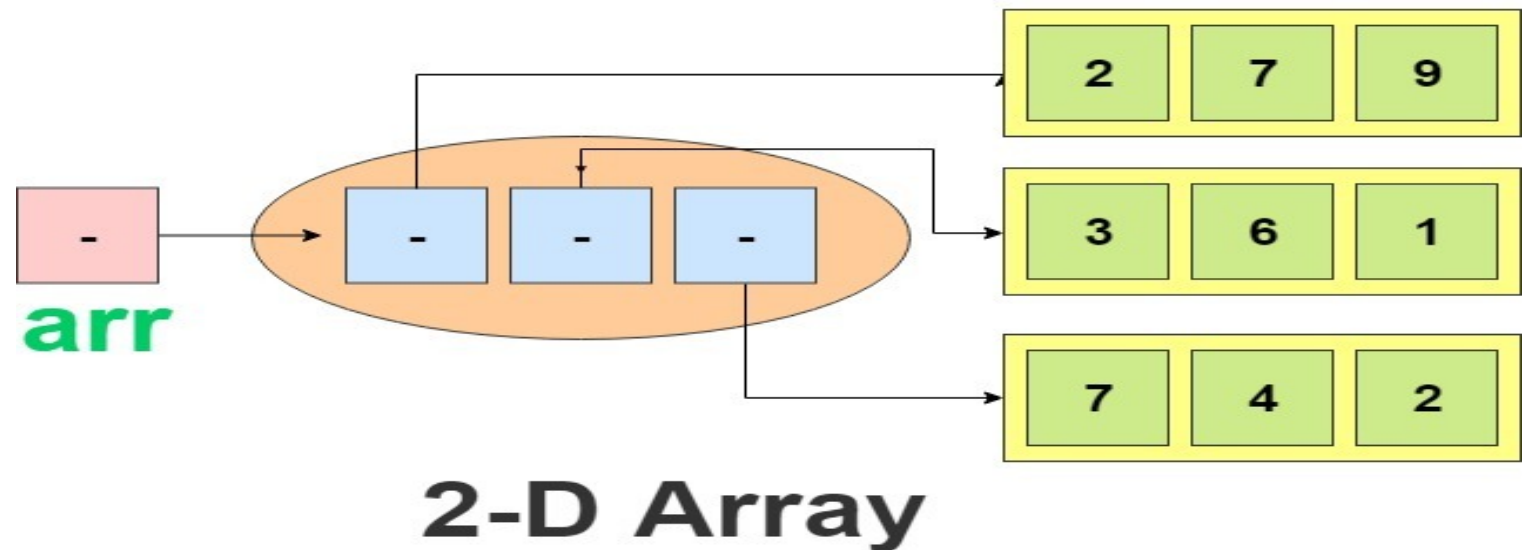
        int month_days[] = { 31, 28, 31, 30, 31, 30, 31, 31, 30, 31,
                             30, 31 };
        System.out.println("April has " + month_days[3] + " days.");
    }
}
```

multi-dimensional array

□ In Java, *multidimensional arrays* are actually arrays of arrays.

□ arr is an 2D Array made up of 3 rows and 3 columns

Multi-Dimensional Array in Java

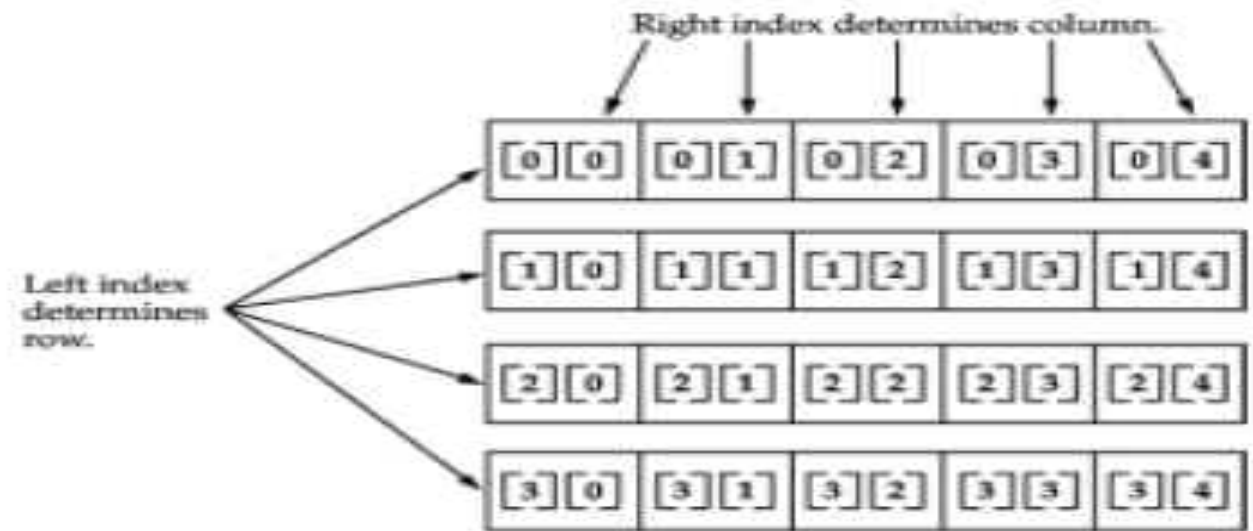


Creating an 2d array

□ **Syntax:** `type arr_name[][]=new type[row_size][column_size];`

□ **Example:** `int twoD[][]=new int [4][5];`

□ Elements are accessed using row and column indices



Given: `int twoD [ ] [ ] = new int [4] [5] ;`

Example : 2d array

□ Write a java program that creates an 2d Array and displays

0	1	2	3	4
5	6	7	8	9
10	11	12	13	14
15	16	17	18	19

Example : 2d array

```
// Demonstrate a two-dimensional array.
class TwoDArray {
    public static void main(String args[]) {
        int twoD[][]= new int[4][5];
        int i, j, k = 0;

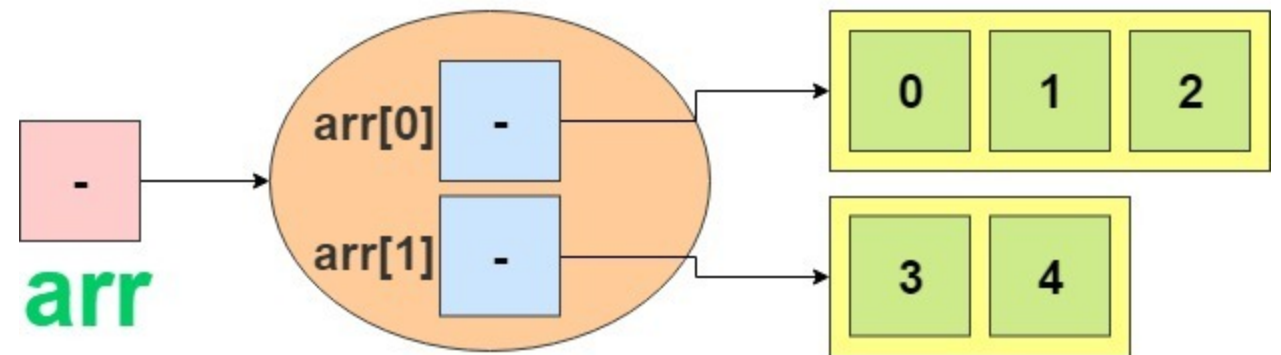
        for(i=0; i<4; i++)
            for(j=0; j<5; j++) {
                twoD[i][j] = k;
                k++;
            }

        for(i=0; i<4; i++) {
            for(j=0; j<5; j++)
                System.out.print(twoD[i][j] + " ");
            System.out.println();
        }
    }
}
```

jagged array

A jagged [array](#) is an array of arrays such that member arrays can be of different sizes, i.e., we can create a 2-D array but with a variable number of columns in each row.

Jagged Array in Java



2-D Array

Creating jagged array

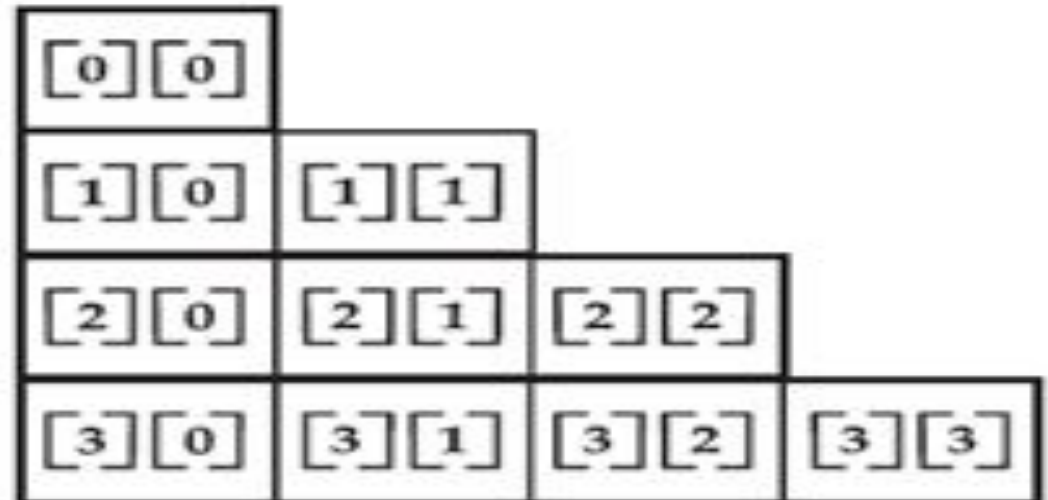
Syntax:

```
data_type array_name[][] = new data_type[n][]; //n: no. of rows  
array_name[0] = new data_type[n1] //n1= no. of columns in row-1  
array_name[1] = new data_type[n2] //n2= no. of colmun in row-2  
array_name[2] = new data_type[n3] //n3= no. of colmun in row-3  
...  
array_name[n-1] = new data_type[nk] //nk=no. of colmun in row-n
```

Creating jagged array

Example:

```
int twoD[][]=new [4][];  
twoD[0]=new int[1];  
twoD[1]=new int[2];  
twoD[2]=new int[3];  
twoD[3]=new int[4];
```



Example : jagged array

□ Write a java program that creates jagged Array and displays

```
0  
1 2  
3 4 5  
6 7 8 9
```

Example : jagged array

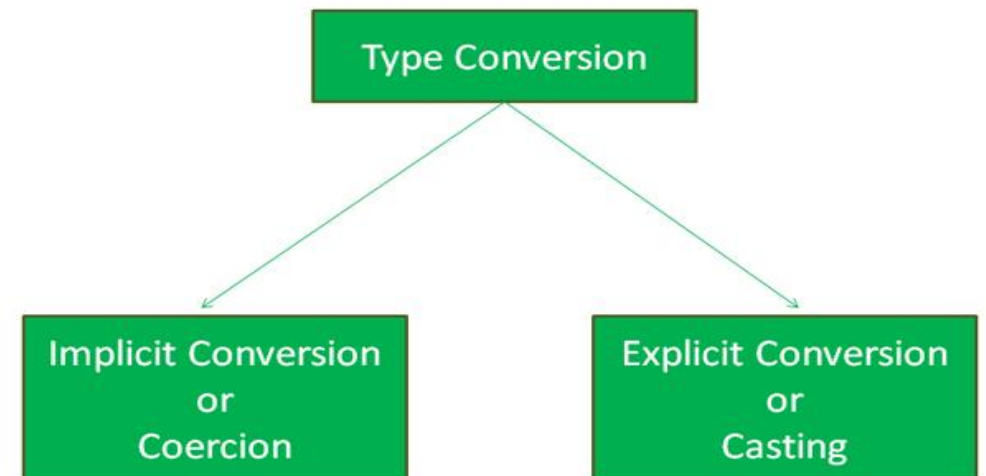
```
class TwoDAgain {  
    public static void main(String args[]) {  
        int twoD[][] = new int[4][];  
        twoD[0] = new int[1];  
        twoD[1] = new int[2];  
        twoD[2] = new int[3];  
        twoD[3] = new int[4];  
  
        int i, j, k = 0;  
  
        for(i=0; i<4; i++)  
            for(j=0; j<i+1; j++) {  
                twoD[i][j] = k;  
                k++;  
            }  
  
        for(i=0; i<4; i++) {  
            for(j=0; j<i+1; j++)  
                System.out.print(twoD[i][j] + " ");  
            System.out.println();  
        }  
    }  
}
```

agenda

- Type Conversion and Casting
- Automatic type Promotions in expressions
- Strings

Type conversions

- When we assign value of one data type to another, the two types might not be compatible with each other.
- If the data types are compatible, then Java will perform the conversion automatically known as Automatic/Implicit Type Conversion and if not then they need to be casted or converted explicitly.
- For example, assigning an int value to a long variable.
- For example, there is no automatic conversion from double to byte, explicit casting to be done.



Java's automatic conversions

- When one type of data is assigned to another type of variable, an *automatic type conversion* will take place if the following two conditions are met:
 - The two types are compatible.
 - The destination type is larger than the source type.
- When these two conditions are met, a *widening conversion* takes place.
- For widening conversions, the numeric types, including integer and floating point types are compatible with each other.
- There are no automatic conversions from the numeric types to **char** or **boolean**.



Java's automatic conversions-example

```
class Test
{
    public static void main(String[] args)
    {
        int i = 100;
        // automatic type conversion
        long l = i;
        // automatic type conversion
        float f = l;
        System.out.println("Int value "+i);
        System.out.println("Long value "+l);
        System.out.println("Float value "+f);
    }
}
```

Casting incompatible types

- We cannot assign a value of larger data type to a smaller data type, For example we cannot assign a int value to a byte variable, because byte is smaller data type than int.
- We need to perform **explicit type casting or narrowing**.
- This is useful for incompatible data types where automatic conversion cannot be done.
- To create a conversion between two incompatible types, you must use a cast.
- A *cast* is simply an explicit type conversion. It has this general form:

(target-type) value

Here, target-type specifies the desired type to convert the specified value to.

Casting incompatible types

//Java program to illustrate Conversion of int and double to byte

```
class Test
```

```
{
```

```
    public static void main(String args[])
```

```
    {
```

```
        byte b;
```

```
        int i = 257;
```

```
        double d = 323.142;
```

```
        System.out.println("Conversion of int to byte.");
```

```
        //i%256
```

```
        b = (byte) i;
```

```
        System.out.println("i = " + i + " b = " + b);
```

```
        System.out.println("\nConversion of double to byte.");
```

```
        //d%256
```

```
        b = (byte) d;
```

```
        System.out.println("d = " + d + " b= " + b);
```

```
    }
```

```
}
```

double → float → long → int → short → byte

Narrowing

Casting incompatible types-example

//Java program to illustrate explicit type conversion

```
class Test
{
    public static void main(String[] args)
    {
        double d = 100.04;
        //explicit type casting
        long l = (long)d;
        //explicit type casting
        int i = (int)l;
        System.out.println("Double value "+d);

        //fractional part lost
        System.out.println("Long value "+l);

        //fractional part lost
        System.out.println("Int value "+i);
    }
}
```

automatic type promotion in expressions

In addition to assignments, there is another place where certain type conversions may occur, in expressions.

Example:

```
byte a=40;  
byte b=50;  
byte c= 100;  
int d=a * b / c;
```

✓ $40 * 50 = 2000$, this exceeds the range of byte operands.

✓ To solve this kind of problem, byte, short, and char values are promoted to int when evaluating an expression.

✓ `byte b=50;`

`b=b*2; //Error! Cannot assign an int to a byte`

`b=(byte)(b*2); //Explicit type casting required`

automatic type promotion in expressions

Java defines several type promotion rules that apply to expressions. They are as follows:

- First, all byte, short, and char values are promoted to int.
- Then, if one operand is a long, the whole expression is promoted to long.
- If one operand is a float, the entire expression is promoted to float.
- If any of the operands is double, the result is double.

automatic type promotion in expressions

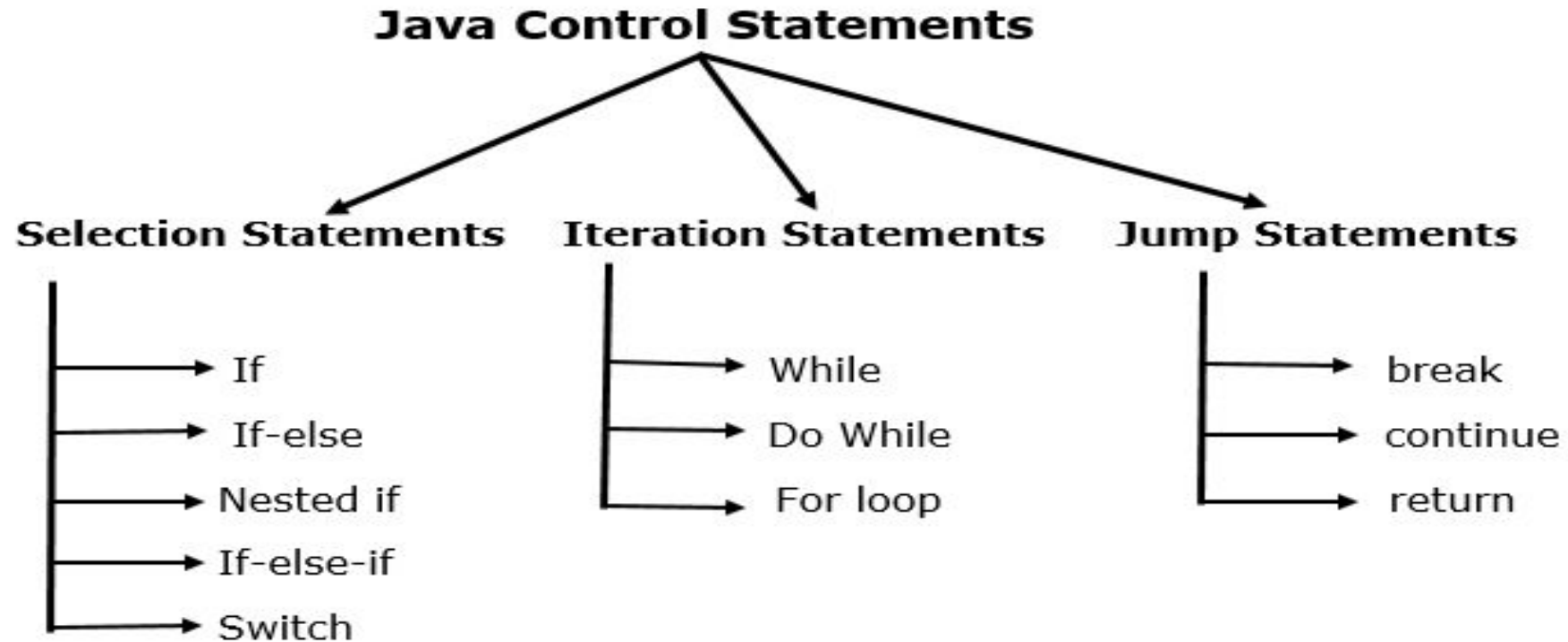
```
class Promote
{
public static void main(String args[])
{
    byte b = 42;
    char c = 'a';
    short s = 1024;
    int i = 50000;
    float f = 5.67f;
    double d = .1234;
    double result = (f * b) + (i / c) - (d * s);
    System.out.println((f * b) + " + " + (i / c) + " - " + (d * s));
    System.out.println("result =" + result);
}
}
```


Strings

- The **String** type is used to declare string variables.
- We can also declare arrays of strings.
- A quoted string constant can be assigned to a **String** variable.
- A variable of type **String** can be assigned to another variable of type **String**.
- We can use an object of type **String** as an argument to **println()**.

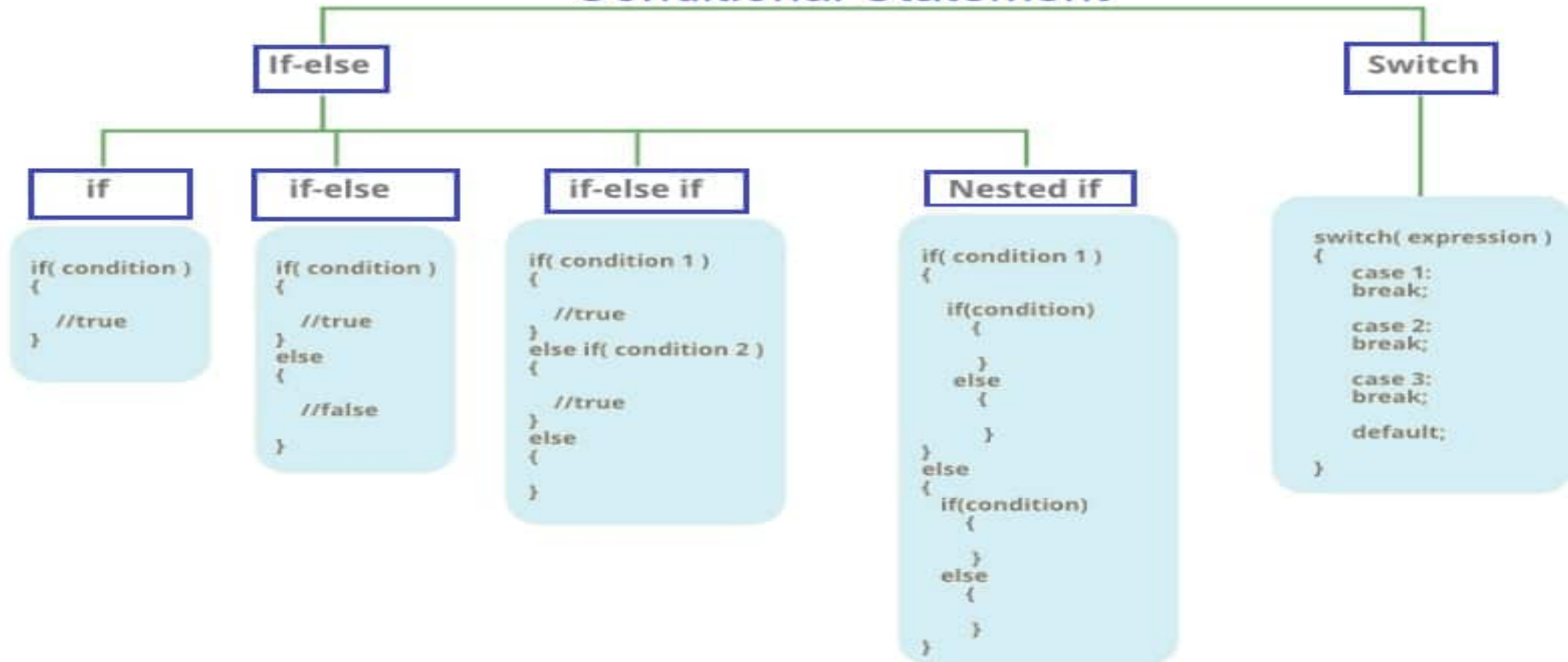
```
String str = "this is a test";  
System.out.println(str);
```

Control statements- introduction

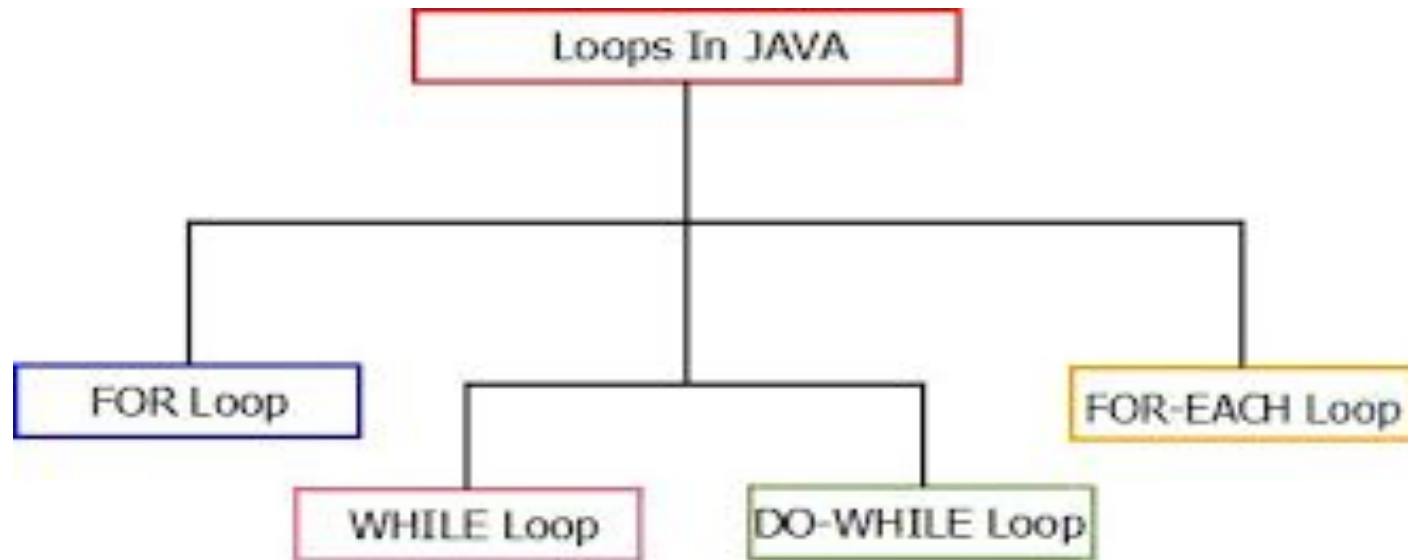


Selection statements

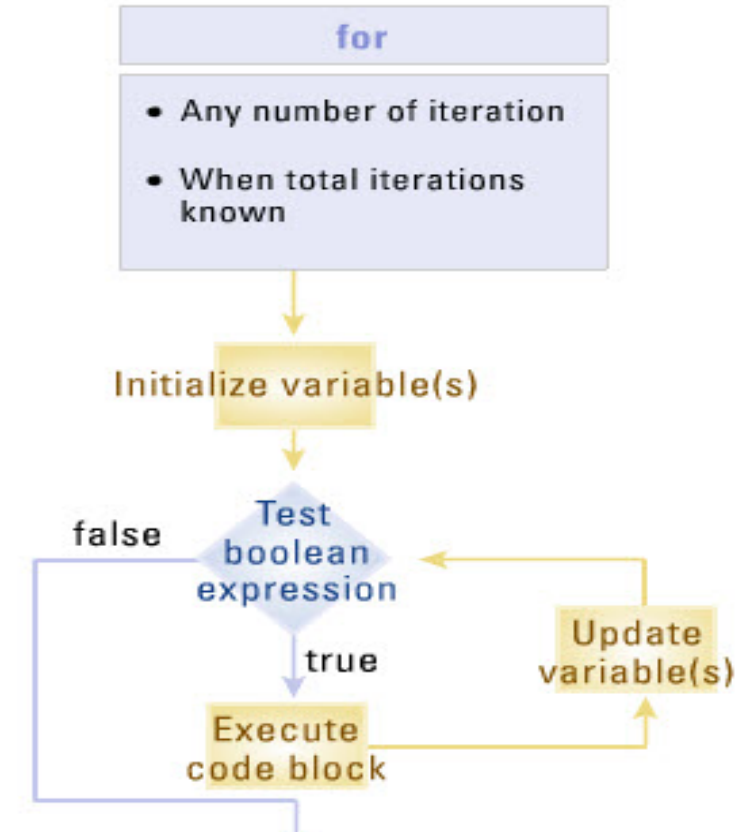
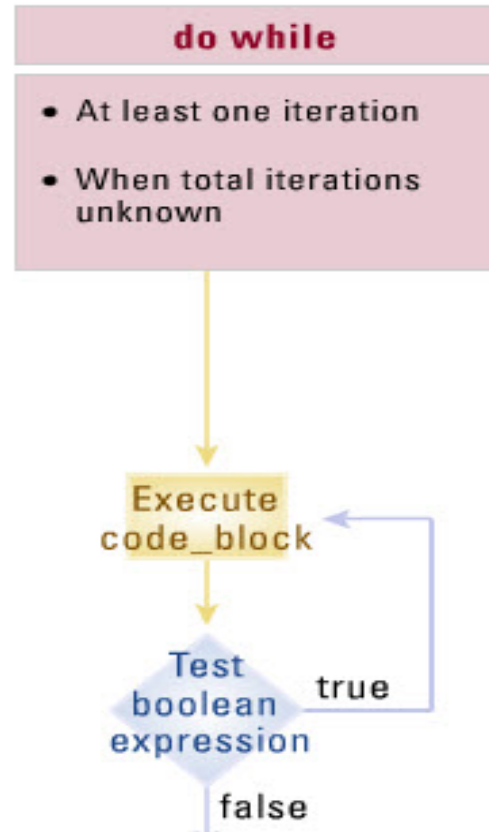
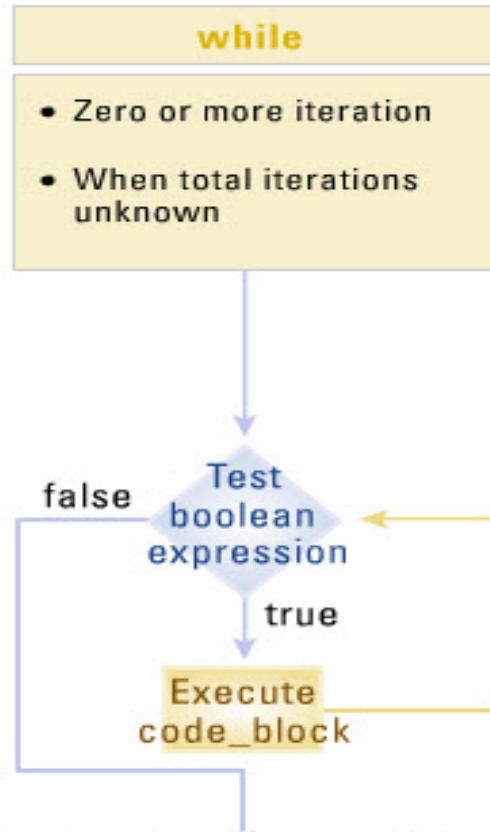
Conditional Statement



iteration statements



iteration statements



Recap..

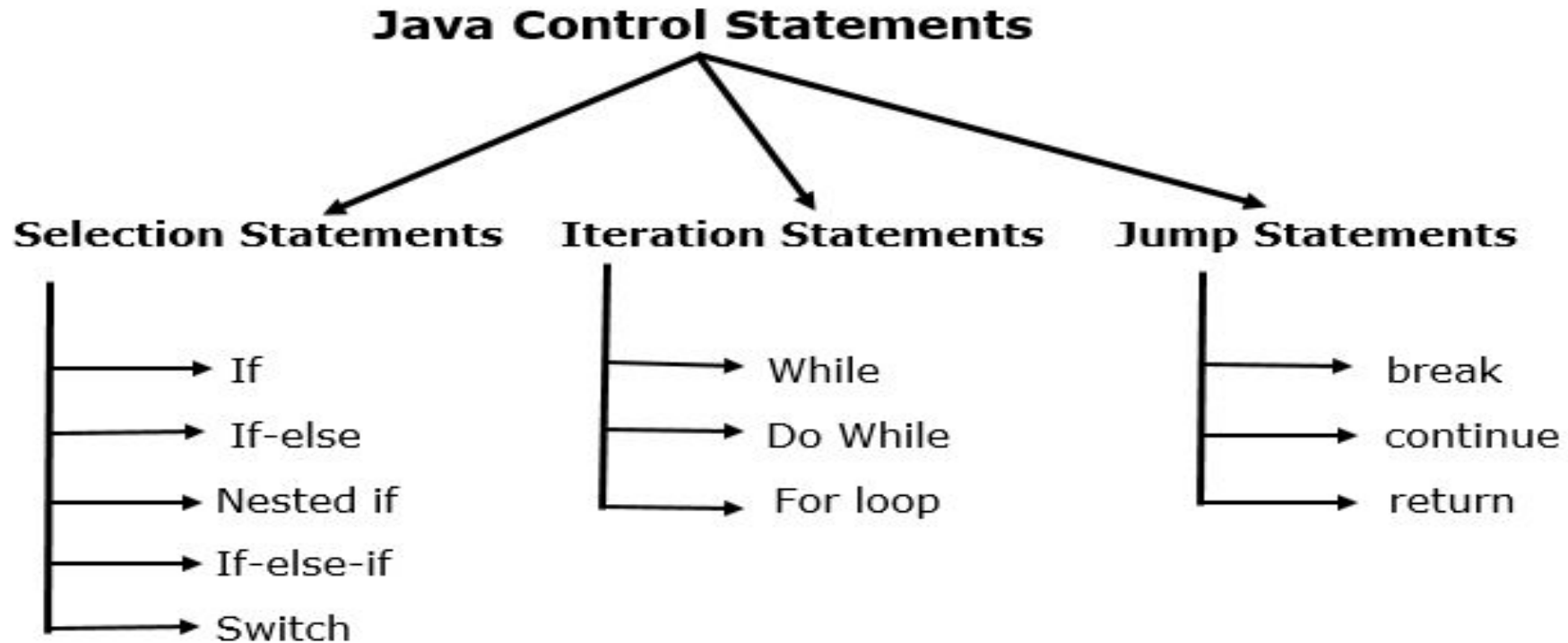
- Operator Precedence
- Using Parentheses
- Introduction to Control Statements

AGENDA

□ Selection Statements

- ✓ if statement
- ✓ if-else statement
- ✓ Nested if statement
- ✓ if-else if statement
- ✓ switch statement

Control statements- introduction



Selection statements- if statement

- In java, we use the if statement to test a condition and decide the execution of a block of statements based on that condition result.
- The if statement checks, the given condition then decides the execution of a block of statements.
- If the condition is **True**, then the block of statements is executed and if it is **False**, then the block of statements is ignored.

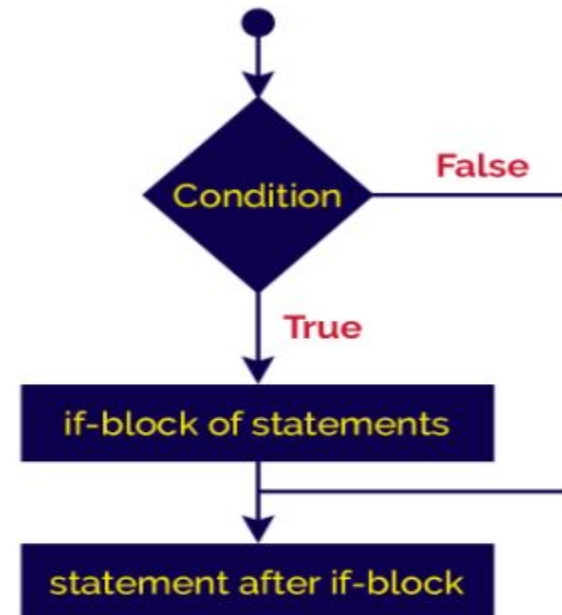
Selection statements- if statement

□ The syntax and execution flow of if the statement is as follows.

Syntax

```
if(condition){  
    if-block of statements;  
    ...  
}  
statement after if-block;  
...
```

Flow of execution



Selection statements- if statement

□Example:

```
import java.util.Scanner;
public class IfStatementTest
{
    public static void main(String[] args)
    {
        Scanner read = new Scanner(System.in);
        System.out.print("Enter any number: ");
        int num = read.nextInt();
        if(num % 5 == 0)
        {
            System.out.println("We are inside the if-block!");
            System.out.println("Given number is divisible by 5!!");
        }
        System.out.println("We are outside the if-block!!!");
    }
}
```

Selection statements- if else statement

- In java, we use the if-else statement to test a condition and pick the execution of a block of statements out of two blocks based on that condition result.
- The if-else statement checks the given condition then decides which block of statements to be executed based on the condition result.
- If the condition is **True**, then the **if block of statements** is executed and if it is **False**, then the **else block of statements** is executed.
- Condition is always an expression that returns a boolean value

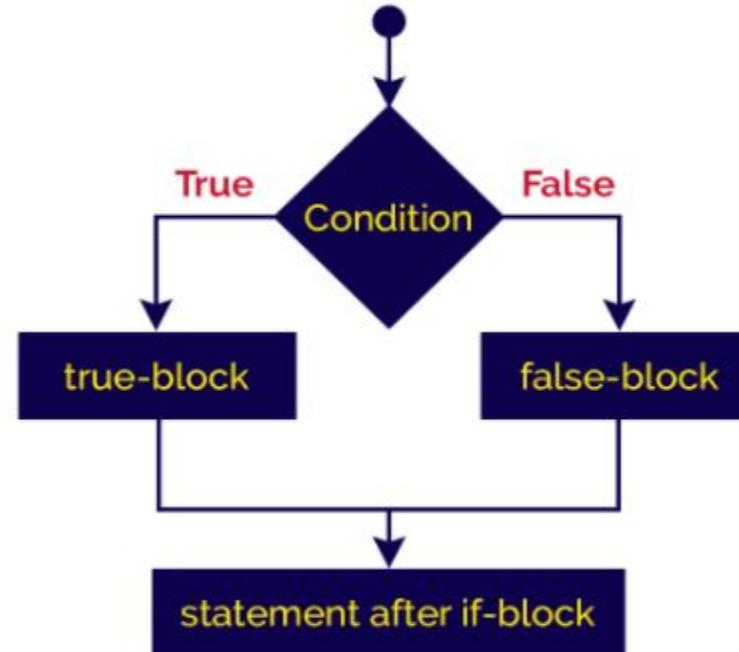
Selection statements- if else statement

□ The syntax and execution flow of if-else statement is as follows.

Syntax

```
if(condition){  
    true-block of statements;  
    ...  
}  
else{  
    false-block of statements;  
    ...  
}  
statement after if-block;  
...
```

Flow of execution



Selection statements- if else statement

Example:

```
import java.util.Scanner;
public class IfElseStatementTest
{
    public static void main(String[] args)
    {
        Scanner read = new Scanner(System.in);
        System.out.print("Enter any number: ");
        int num = read.nextInt();
        if((num % 2) == 0)
        {
            System.out.println("We are inside the true-block!");
            System.out.println("Given number is EVEN number!!");
        }
        else
        {
            System.out.println("We are inside the false-block!");
            System.out.println("Given number is ODD number!!");
        }
        System.out.println("We are outside the if-block!!!");
    }
}
```

Selection statements- nested if statements

- Writing an if statement inside another if-statement is called nested if statement.
- The general syntax of the nested if-statement is as follows.

Syntax

```
if ( condition1 )  
{  
    if ( condition2 )  
    {  
        ....  
        True block of statements 1;  
    }  
    ....  
}  
else  
{  
    False block of condition1;  
}
```

Selection statements- nested if statements

□Example:

```
import java.util.Scanner;
public class NestedIfStatementTest
{
    public static void main(String[] args)
    {
        Scanner read = new Scanner(System.in);
        System.out.print("Enter any number: ");
        int num = read.nextInt();
        if (num < 100)
        {
            System.out.println("\nGiven number is below 100");
            if (num % 2 == 0)
                System.out.println("And it is EVEN");
            else
                System.out.println("And it is ODD");
        }
        else
            System.out.println("Given number is not below 100");
        System.out.println("We are outside the if-block!!!");
    }
}
```

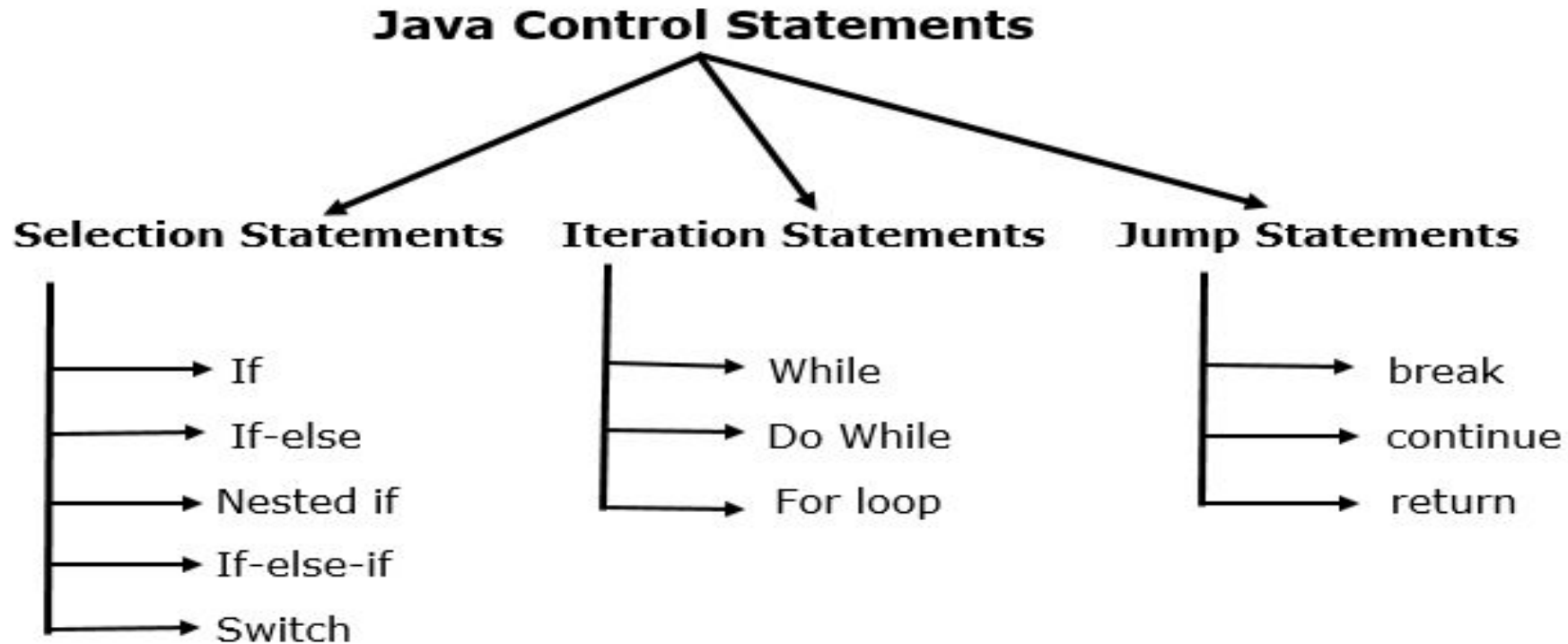

assignment

- Program to check whether a person is eligible to vote or not.
- Program to check whether the given number is positive or negative
- Program to check whether the given character is vowel or not
- Program to check whether the entered year is a leap year or not.
- Program to check whether the person is eligible to be an employee or not.
- Program to find largest of three numbers using nested if statements

AGENDA

- Selection Statements
 - ✓ if-else if statement
 - ✓ switch statement
 - ✓ Example Programs
 - ✓ Assignment

Control statements- introduction

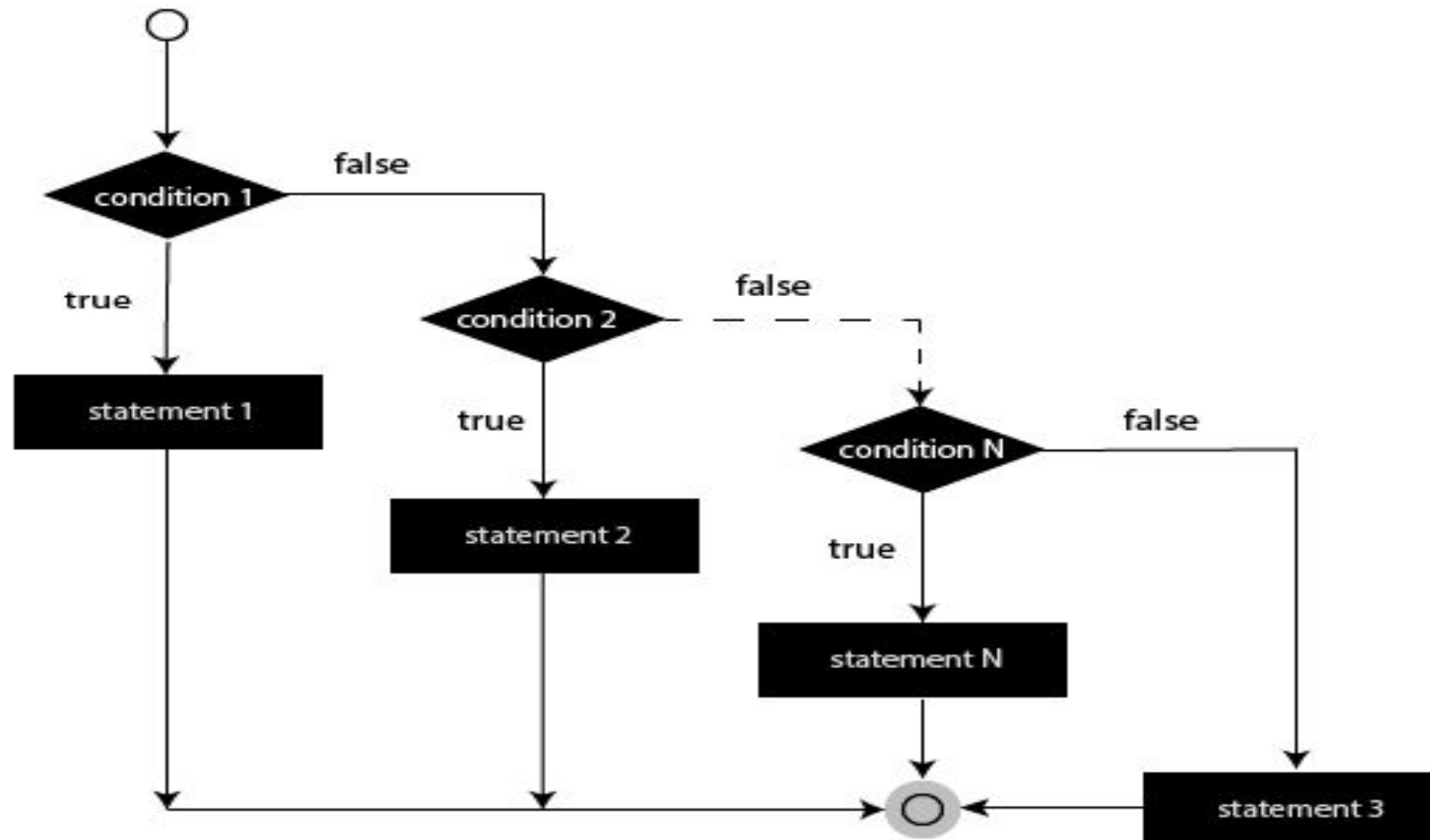


Selection statements- if-else if statements

- In Java, we have an **if...else...if** ladder, that can be used to execute one block of code among multiple other blocks.
- The general syntax of the nested if-statement is as follows.
- Here, if statements are executed from the top towards the bottom. When the test condition is **true**, codes inside the body of that **if block** is executed. And, program control jumps outside the if...else...if ladder.
If all test expressions are **false**, codes inside the **body of else** are executed.

```
if (condition1) {  
    // codes  
}  
else if(condition2) {  
    // codes  
}  
else if (condition3) {  
    // codes  
}  
.  
.  
else {  
    // codes  
}
```

Selection statements- if-else if statements

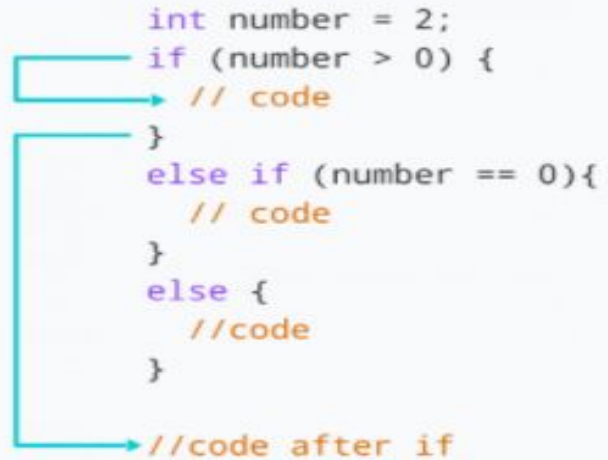


Selection statements- if-else if statements

How the if...else...if ladder works?

1st Condition is true

```
int number = 2;  
if (number > 0) {  
    // code  
}  
else if (number == 0){  
    // code  
}  
else {  
    //code  
}  
//code after if
```



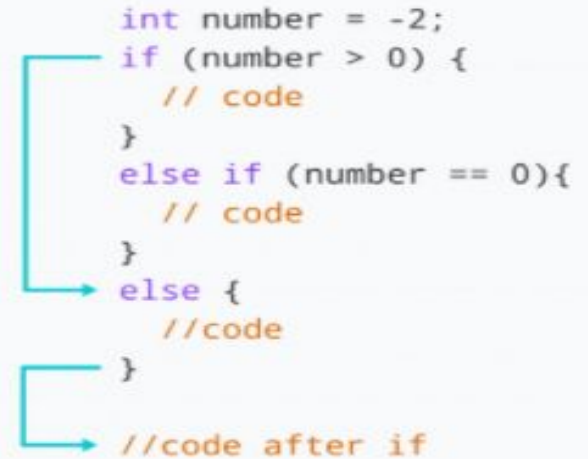
2nd Condition is true

```
int number = 0;  
if (number > 0) {  
    // code  
}  
else if (number == 0){  
    // code  
}  
else {  
    //code  
}  
//code after if
```



All Conditions are false

```
int number = -2;  
if (number > 0) {  
    // code  
}  
else if (number == 0){  
    // code  
}  
else {  
    //code  
}  
//code after if
```



Selection statements- if-else if statements

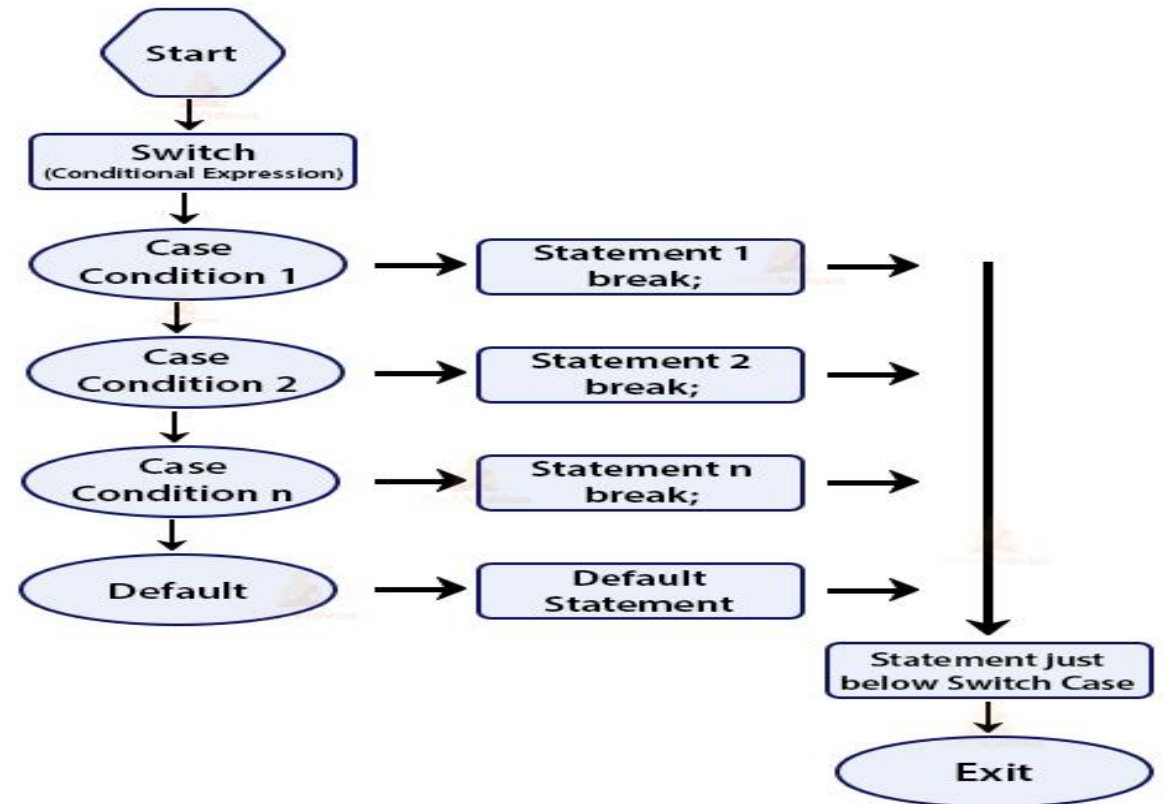
```
class Main
{
    public static void main(String[] args)
    {
        int number = 0;
        // checks if number is greater than 0
        if (number > 0)
        {
            System.out.println("The number is positive.");
        }
        // checks if number is less than 0
        else if (number < 0)
        {
            System.out.println("The number is negative.");
        }
        // if both condition is false
        else
        {
            System.out.println("The number is 0.");
        }
    }
}
```

Write a JAVA Program to find largest of three numbers using else if ladder.

Selection statements- switch statement

- Use the switch statement to select one of many code blocks to be executed.
- This is how it works:
- The switch expression is evaluated once.
- The value of the expression is compared with the values of each case.
- If there is a match, the associated block of code is executed.
- Else default block will be executed
- The expression must be of type byte, short, int or char

Switch Statement in Java



Selection statements- switch statement

Syntax:

```
switch(expression){  
  case value1:  
    //code to be executed;  
    break; //optional  
  case value2:  
    //code to be executed;  
    break; //optional  
  .....  
  default:  
    code to be executed if all cases are not matched;  
}
```

Selection statements- switch statement

```
public class SwitchExample {  
    public static void main(String[] args) {  
        //Declaring a variable for switch expression  
        int number=20;  
        //Switch expression  
        switch(number){  
            //Case statements  
            case 10: System.out.println("10");  
            break;  
            case 20: System.out.println("20");  
            break;  
            case 30: System.out.println("30");  
            break;  
            //Default case statement  
            default: System.out.println("Not in 10, 20 or 30");  
        }  
    }  
}
```

Write a Java Program that simulates simple calculator using switch case statement
<https://www.softwaretestinghelp.com/java-switch-case-statement/>

assignment

- Program to read weekday number and print weekday name using switch
- Program to design calculator with basic operations(+, -, *, /, %) using switch
- Program to check whether the given point(x,y) lies in I, II , III and IV Quadrant
- Program to check the type of triangle by reading its three sides

AGENDA

□ Selection Statements

- ✓ Switch statement – Example Programs

□ Iteration Statements

- ✓ While
- ✓ Do-While Statement
- ✓ Example
- ✓ Assignment

Selection statements- switch statement

public class example

```
{  
    public static void main(String[] args)  
    {  
        for(int i=0; i<5; i++)  
        {  
            switch(i)  
            {  
                case 0:  
                    System.out.println("i value is 0");  
                    break;  
                case 1:  
                    System.out.println("i value is 1");  
                    break;  
                case 2:  
                    System.out.println("i value is 2");  
                    break;  
            }  
        }  
    }  
}
```

default:

```
    System.out.println("i value is greater than 2 and less than 5");  
}
```

OUTPUT:

```
i value is 0  
i value is 1  
i value is 2  
i value greater than 2 and less than 5  
i value greater than 2 and less than 5
```

Selection statements- switch statements

- In Switch case, the **break statement** is optional.
- When we omit the break the control of the program i.e., execution will flow to the next case.
- This allows us to have multiple cases with one set of code to be executed
- This is referred to as **fall through**- i.e., the execution falls through each case until a break statement is encountered or the end of the switch is reached

```
switch (month) {  
    case 1:  
    case 3:  
    case 5:  
    case 7:  
    case 8:  
    case 10:  
    case 12:  
        numberOfDays = 31;  
        break;  
  
    case 4:  
    case 6:  
    case 9:  
    case 11:  
        numberOfDays = 30;  
        break;  
  
    case 2:  
        System.out.print("Leap year (true/false)? ");  
        isLeapYear = myScanner.nextBoolean();  
        if (isLeapYear)  
            numberOfDays = 29;  
        else  
            numberOfDays = 28;  
}  
System.out.print(numberOfDays);  
System.out.println(" days");
```

Selection statements- switch statement

```
public class example {
```

```
    public static void main(String[] args)
    {
```

```
        for(int i=0;i<=10; i++)
```

```
        {
```

```
            switch(i)
```

```
            {
```

```
                case 0:
```

```
                case 1:
```

```
                case 2:
```

```
                case 3:
```

```
                case 4:
```

```
                    System.out.println("i value is less than 5");
```

```
                    break;
```

```
                case 5:
```

```
                case 6:
```

```
                case 7:
```

```
                case 8:
```

```
                case 9:
```

```
                    System.out.println("i value is less than 10");
```

```
                    break;
```

```
                default:
```

```
                    System.out.println("Default statement");
```

```
            }
```

```
        }
```

```
    }
```

```
}
```

Selection statements- nested switch statements

- This involves the concept of an inner and outer Switch.
- We can use an inner Switch as a part of the statement of an outer Switch.
- This type of Switch statement is called the Nested Switch statement
- In the example, **case 1:** statement in the **inner switch** does not conflict with the **case 1:** statement in the outer switch.
- The count variable is compared with the list of cases at the outer level, and when count is 1, then the target is compared with the inner list of cases

```
switch (count)
{
    case 1:
        switch (target)
        {
            //nested switch statement
            case 0:
                System.out.println("target is 0");
                break;

            case 1:
                System.out.println("target is 1");
                break;

        }
        break;
    case 2:
        //...
}
```


Selection statements- switch statement

```
public class ExampleOfNestedSwitch
{
    public static void main(String[] args)
    {
        int year = 3;
        int marks = 80;
        switch(year)
        {
            case 1:
                System.out.println("First year students are
                not eligible for scholarship ");
                break;
            case 2:
                System.out.println("Second year students
                are not eligible for scholarship");
                break;
```

```
        case 3:
            switch(marks)
            {
                case 50:
                    System.out.println("You are not eligible
                    for scholarship");
                    break;
                case 80:
                    System.out.println("Congrats!!!! you are
                    eligible for scholarship");
                    break;
            }
            break;
        default: System.out.println("Please enter valid year");
    }
}
```

Iteration statements- while statement

- The while statement is used to execute a single statement or block of statements repeatedly as long as the given condition is TRUE.
- When the condition becomes FALSE, control passes to the next line of code immediately following the while loop
- The condition can be any Boolean expression
- The while statement is also known as Entry control looping statement.

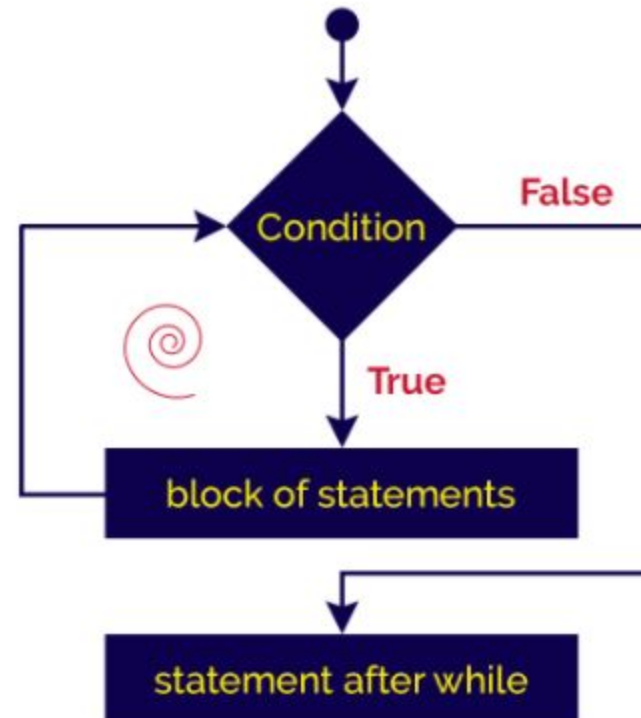
Iteration statements- while

□ The syntax and execution flow of while statement is as follows.

Syntax

```
while(boolean-expression){  
    block of statements;  
    ...  
}  
statement after while;  
...
```

Flow of execution



Iteration statements- while

□Example:

```
public class WhileTest
{
    public static void main(String[] args)
    {
        int num = 1;
        while(num <= 10)
        {
            System.out.println(num);
            num++;
        }
        System.out.println("Statement after while!");
    }
}
```

OUTPUT:

```
1
2
3
4
5
6
7
8
9
10
Statement after while!
```

Iteration statements- do while statement

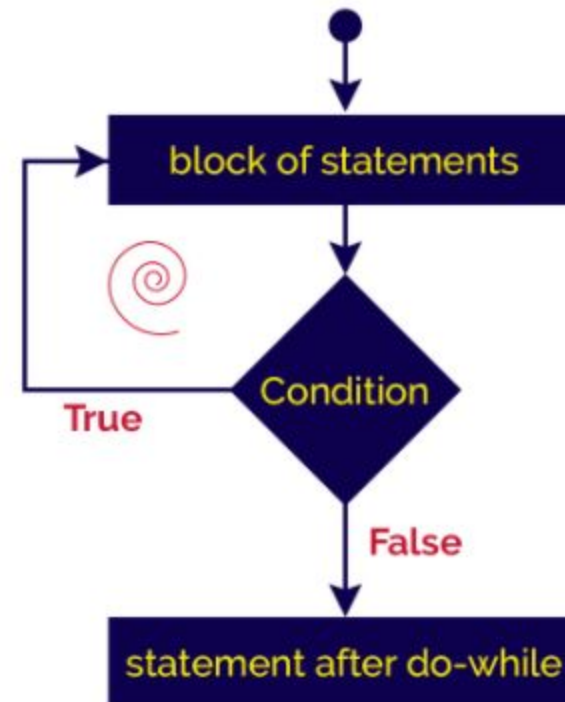
- While loop do checks for the loop condition at the beginning of the loop. Unless the condition is satisfied i.e., true the loop will not be executed.
- The do-while loop is mainly used in the case where we need to execute the loop at least once, even if the conditional expression is false to begin with.
- The do while loop checks the condition at the end of the loop. This means that the statements inside the loop body will be executed at least once even if the condition is never true.
- The do-while statement is used to execute a single statement or block of statements repeatedly as long as the given condition is TRUE.
- The do-while statement is also known as Exit control looping statement.

Iteration statements- do while statement

Syntax

```
do{  
    block of statements;  
}while(boolean-expression);  
statement after do-while;  
...
```

Flow of execution



Iteration statements- do while statement

```
public class DoWhileTest
{
    public static void main(String[] args)
    {
        int num = 1;
        do
        {
            System.out.println(num);
            num++;
        }while(num <= 10);
        System.out.println("Statement after do-while!");
    }
}
```

OUTPUT:

```
1
2
3
4
5
6
7
8
9
10
Statement after while!
```

Iteration statements- do while statement

```
import java.util.Scanner;
public class MenuDrivenSystem
{
public static void main (String[] args)
{
    float a,b,area = 0;
    int ch;
    Scanner input = new Scanner(System.in);
    do
    {
        System.out.println("Enter 1 for circle.");
        System.out.println("Enter 2 for square.");
        System.out.println("Enter 3 for rectangle.");
        ch = input.nextInt();
        switch(ch)
        {
            case 1: //code
```

```
            case 2: //code
            case 3: //code
            default: System.out.println("Error");
        }
        System.out.println("Area = "+area);
    }while(ch>=1 && ch<=3);
}
}
```


AGENDA

□ Iteration Statements

- ✓ for loop
- ✓ foreach loop
- ✓ Example
- ✓ Assignment

Iteration statements- for loop

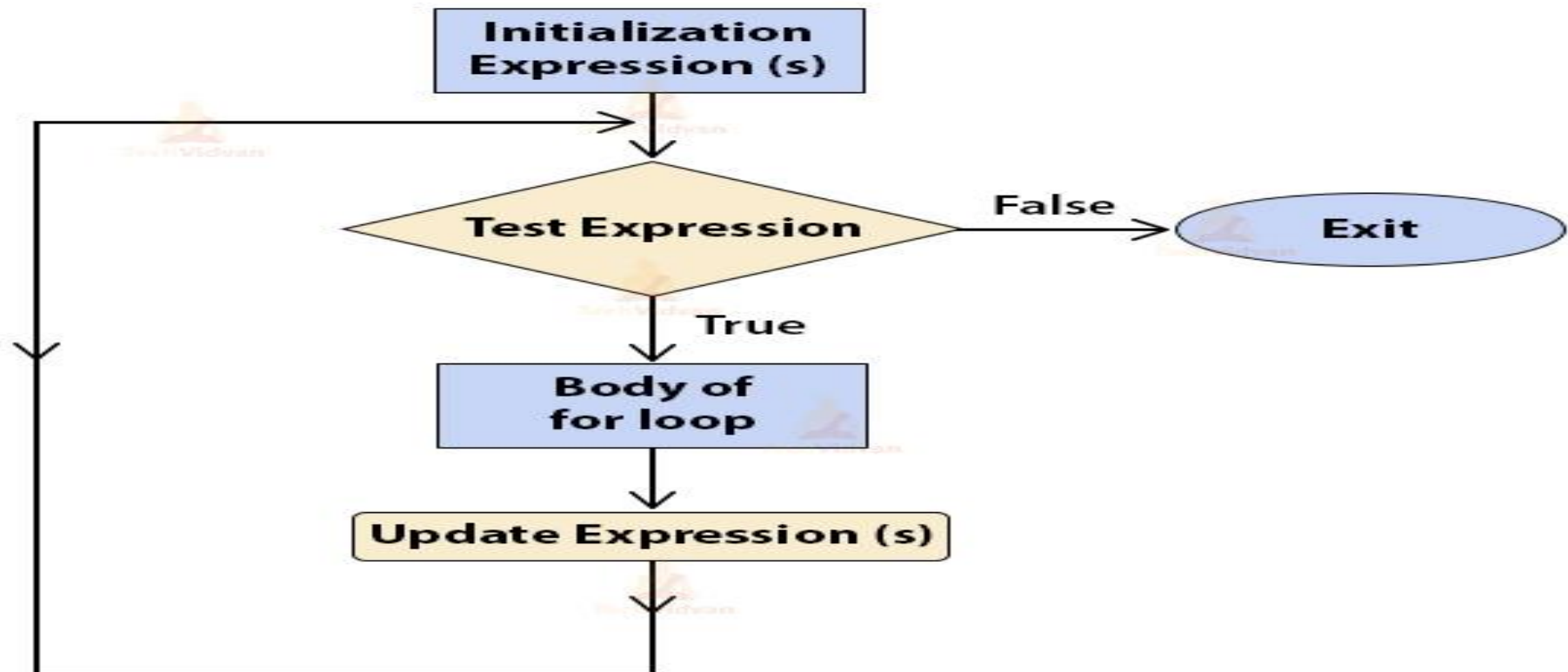
- The for loop is a looping construct which can execute a set of instructions a specified number of times.
- It's a counter controlled loop.

Syntax:

```
for (<initialization>; <loop condition>; <increment expression>)  
{  
    <loop body>  
}
```

Iteration statements- for loop

The for Loop in Java



Iteration statements- for loop

```
public class Example
{
    public static void main(String[] args)
    {
        System.out.println("Printing Numbers from 1 to 10");
        for (int count = 1; count <= 10; count++)
        {
            System.out.println(count);
        }
    }
}
```

Iteration statements- for loop

Declaring Loop Control Variables Inside the for Loop

- Often the variable that controls a **for** loop is only needed for the purposes of the loop and is not used elsewhere.
- When this is the case, it is possible to declare the variable inside the initialization portion of the **for**.
- Example:

```
class ForTick
{
    public static void main(String args[])
    {
        // here, n is declared inside of the for loop
        for(int n=10; n>0; n--)
            System.out.println("tick " + n);
    }
}
```

- When you declare a variable inside a **for** loop, there is one important point to remember: the scope of that variable ends when the **for** statement does

Iteration statements- for loop

Using the Comma

□ There will be times when you will want to include more than one statement in the initialization and iteration portions of the **for** loop.

□ Example

```
class Comma
{
    public static void main(String args[])
    {
        int a, b;
        for(a=1, b=4; a<b; a++, b--)
        {
            System.out.println("a = " + a);
            System.out.println("b = " + b);
        }
    }
}
```

Iteration statements- for loop

Some for Loop Variations

- The **for** loop supports a number of variations that increase its power and applicability.
- The reason it is so flexible is that its three parts—the initialization, the conditional test, and the iteration—do not need to be used for only those purposes can be used for any purpose you desire.
- One of the most common variations involves the conditional expression.
- Specifically, this expression does not need to test the loop control variable against some target value.

Iteration statements- for loop

Some for Loop Variations

□ In fact, the condition controlling the **for** can be any Boolean expression.

□ For example, consider the following fragment:

```
boolean done = false;
for(int i=1; !done; i++)
{
    // ...
    if(interrupted())
        done = true;
}
```

□ In this example, the **for** loop continues to run until the **boolean** variable **done** is set to **true**.

□ It does not test the value of **i**.

Iteration statements- for loop

Some for Loop Variations

- Here is another interesting **for** loop variation.
- Either the initialization or the iteration expression or both may be absent, as in this next program:

// Parts of the for loop can be empty.

```
class ForVar
{
public static void main(String args[])
{
    int i;
    boolean done = false;
    i = 0;
    for( ; !done; )
    {
        System.out.println("i is " + i);
        if(i == 10)
            done = true;
        i++;
    }
}
}
```

Iteration statements- for loop

Some for Loop Variations

- Here is one more **for** loop variation.
- You can intentionally create an infinite loop (a loop that never terminates) if you leave all three parts of the **for** empty.
- For example:

```
for( ; ; )  
{  
    // ...  
}
```
- This loop will run forever because there is no condition under which it will terminate.

Iteration statements- for each loop

```
class ForEach
{
    public static void main(String args[])
    {
        int nums[] = { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 };
        int sum = 0;
        for(int x : nums)
        {
            sum += x;
        }
        System.out.println("Summation: " + sum);
    }
}
```

Iteration statements- for each loop

```
class ForEach
{
    public static void main(String args[])
    {
        int nums[] = { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 };
        int sum = 0;
        for(int i=0;i<nums.length;i++)
        {
            sum = sum+nums[i];
        }
        System.out.println("Summation: " + sum);
    }
}
```

Iteration statements- for each loop

Java program to search given key element

```
class Search
{
    public static void main(String args[])
    {
        int nums[] = { 6, 8, 3, 7, 5, 6, 1, 4 };
        int val = 5;
        boolean found = false;
        // use for-each style for to search nums for val
        for(int x : nums)
        {
            if(x == val)
            {
                found = true;
                break;
            }
        }
        if(found)
            System.out.println("Value found!");
    }
}
```

assignment

Use Foreach loop statements:

- Write a Program that prints sum of odd and even elements in an array
- Write a Program that finds max element in an array
- Write a Program to performs linear search on given array.

AGENDA

□ Iteration Statements

- ✓ foreach loop-2D Array
- ✓ Nested for loops
- ✓ Example

□ Jump Statements-Break Statement

Iteration statements- for each loop

■ A second form of **for** was defined that implements a “for-each” style loop.

■ The general form of the for-each version of the **for** is shown here:

```
for(type itr-var : collection)  
    statement-block
```

■ Here, *type* specifies the type and *itr-var* specifies the name of an *iteration variable* that will receive the elements from a collection, one at a time, from beginning to end.

■ The collection being cycled through is specified by *collection*.

■ There are various types of collections that can be used with the **for**, but the only type used in this chapter is the array.

Iteration statements- for each loop

Iterating Over Multidimensional Arrays

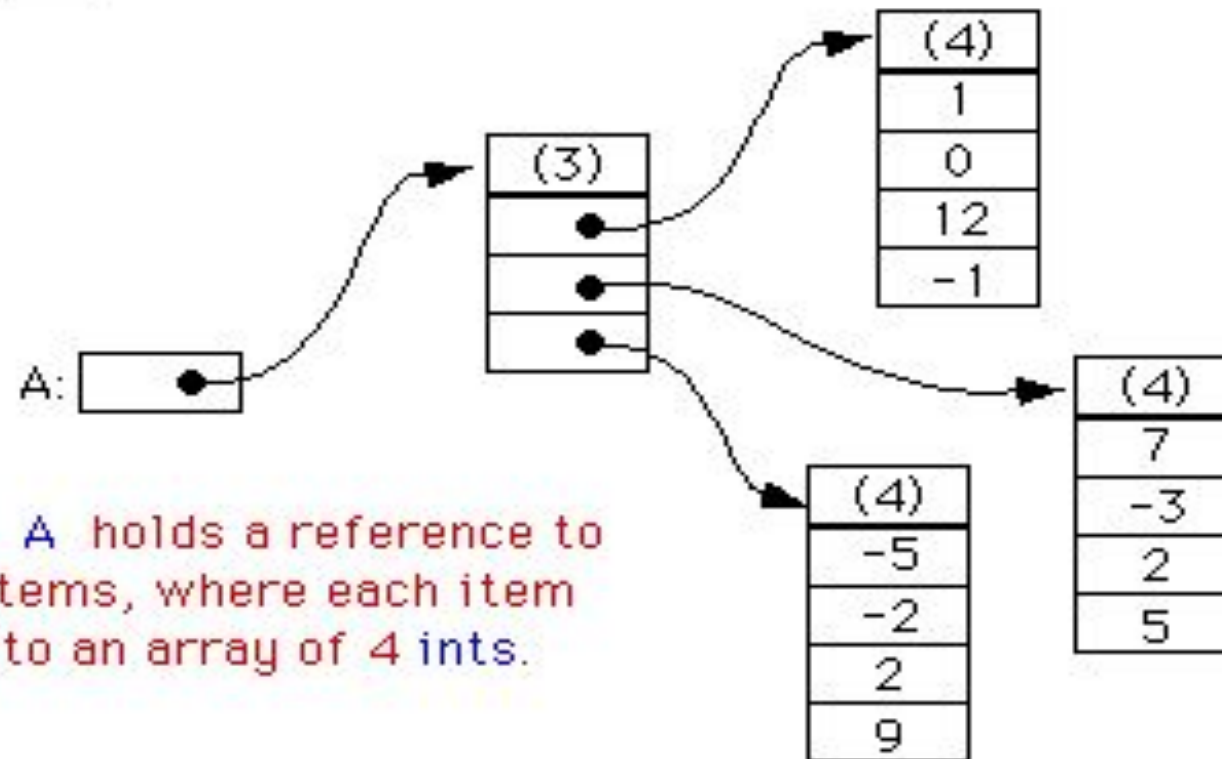
- The enhanced version of the **for** also works on multidimensional arrays.
- Remember, however, that in Java, multidimensional arrays consist of *arrays of arrays*. (For example, a two-dimensional array is an array of one-dimensional arrays.)
- Syntax:

```
for (type variable[] : collection_name)
{
    for(type element:variable)
    {
        statement block
    }
}
```

A:

1	0	12	-1
7	-3	2	5
-5	-2	2	9

If you create an array `A = new int[3][4]`, you should think of it as a "matrix" with 3 rows and 4 columns.



But in reality, `A` holds a reference to an array of 3 items, where each item is a reference to an array of 4 ints.

Iteration statements- for each loop

```
class ForEach3
{
    public static void main(String args[])
    {
        int sum = 0;
        int nums[][] = new int[3][5];
        for(int i = 0; i < 3; i++)
            for(int j=0; j < 5; j++)
                nums[i][j] = (i+1)*(j+1);
        // use for-each for to display and sum the values
        for(int x[] : nums)
        {
            for(int y : x)
            {
                sum += y;
            }
        }
        System.out.println("Summation: " + sum);
    }
}
```

- In the program, pay special attention to this line:
for(int x[] : nums)
- Notice how **x** is declared. It is a reference to a one-dimensional array of integers.
- This is necessary because each iteration of the **for** obtains the next *array* in **nums**, beginning with the array specified by **nums[0]**.
- The inner **for** loop then cycles through each of these arrays, displaying the values of each element.

Iteration statements- nested for loop

```
class Nested
{
    public static void main(String args[])
    {
        int i, j;
        for(i=0; i<10; i++)
        {
            for(j=i; j<10; j++)
                System.out.print("*");
            System.out.println();
        }
    }
}
```

- Like all other programming languages, Java allows loops to be nested.
- That is, one loop may be inside another.

jump statements

- Java supports three jump statements: **break**, **continue**, and **return**.
- These statements transfer control to another part of the program.

Iteration statements- jump statements

The break statement

- The **break** statement transfers control out of the enclosing loop (for, while, do or switch statement).
- We can use a **break** statement when we want to jump immediately to the statement following the enclosing control structure.
- We can also provide a loop with a label, and then use the label in your **break** statement.
- The label name is optional, and is usually only used when we wish to terminate the outermost loop in a series of nested loops.

Syntax:

```
break; // the unlabeled form  
break <label>; // the labeled form
```

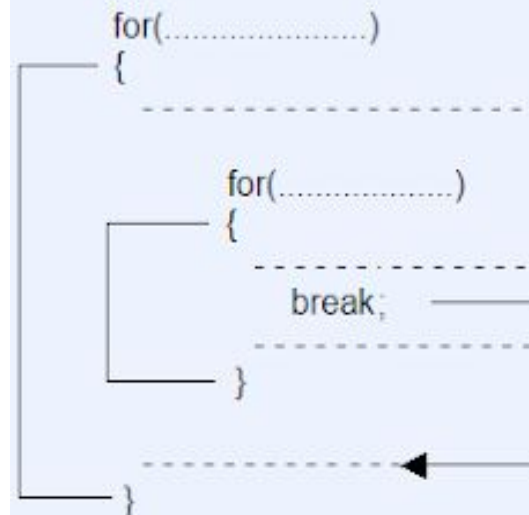
Iteration statements- jump statements

```
while (testExpression) {  
    // codes  
    if (condition to break) {  
        break;  
    }  
    // codes  
}
```

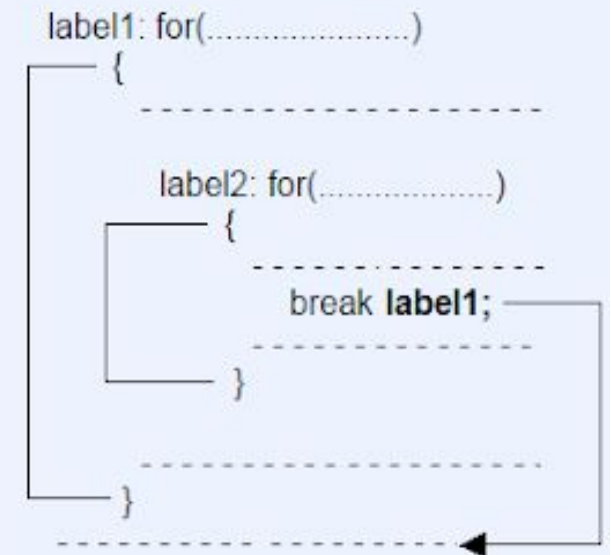
```
do {  
    // codes  
    if (condition to break) {  
        break;  
    }  
    // codes  
} while (testExpression);
```

```
for (init; testExpression; update) {  
    // codes  
    if (condition to break) {  
        break;  
    }  
    // codes  
}
```

Loop without label



Loop with label



Iteration statements- jump statements

```
public class Example
{
    public static void main(String[] args)
    {
        System.out.println("Numbers : ");
        for (int i = 1; ; ++i)
        {
            if (i == 11)
                break;
            System.out.println(i);
        }
        System.out.println("Terminated-Break");
    }
}
```

OUTPUT:

Numbers :

1

2

3

4

5

6

7

8

9

10

Terminated-Break

Iteration statements- jump statements

```
class LabeledBreak
{
    public static void main(String[] args)
    {
        // the for loop is labeled as first
        first: for( int i = 1; i < 5; i++)
        {
            // the for loop is labeled as second
            second: for(int j = 1; j < 3; j ++ )
            {
                System.out.println("i = " + i + "; j = " +j);
                // the break statement breaks the first for loop
                if ( i == 2)
                    break first;
            }
        }
    }
}
```

OUTPUT:

i = 1; j = 1

i = 1; j = 2

i = 2; j = 1

Iteration statements- jump statements

```
class LabeledBreak
{
    public static void main(String[] args)
    {
        // the for loop is labeled as first
        first: for( int i = 1; i < 5; i++)
        {
            // the for loop is labeled as second
            second: for(int j = 1; j < 3; j ++ )
            {
                System.out.println("i = " + i + "; j = " +j);
                // the break statement breaks the first for loop
                if ( i == 2)
                    break;
            }
        }
    }
}
```

OUTPUT:

i = 1; j = 1

i = 1; j = 2

i = 2; j = 1

i = 3; j = 1

i = 3; j = 2

i = 4; j = 1

i = 4; j = 2

Iteration statements- jump statements

```
class Break
{
    public static void main(String args[])
    {
        boolean t = true;
        first: {
            second: {
                third: {
                    System.out.println("Before the break.");
                    if(t)
                        break second; // break out of second block
                    System.out.println("This won't execute");
                }
                System.out.println("This won't execute");
            }
            System.out.println("This is after second block.");
        }
    }
}
```

OUTPUT:

Before the break.

This is after second block.

assignment

Use Nested loop statements to print the below patterns:

```
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
```

Half Pyramid

```
1 2 3 4 5
1 2 3 4
1 2 3
1 2
1
```

Inverted
Half Pyramid

```
*
**
***
****
*****
*****
*****
*****
*****
*****
*****
```

Right Triangle Star
Pattern

```
* * * * *
 * * * * *
  * * * * *
   * * * * *
    * * * * *
     * * * * *
      * * * *
       * * *
        * *
         *
```

Inverted Mirrored Right
Triangle Star Pattern

AGENDA

- Jump Statements
 - ✓ Continue Statement
 - ✓ Return Statement
 - ✓ Examples
 - ✓ Assignment

jump statements- continue statement

The continue statement

- The **continue** keyword can be used in any of the loop control structures.
- When encountered the compiler causes the current iteration to terminate and immediately jump to the next iteration of the loop.
- In a for loop, the continue keyword causes control to immediately jump to the iteration statement and then to the conditional expression
- In a while loop or do/while loop, control immediately jumps to the conditional expression.

jump statements- continue statement

The continue statement

- We can also provide a loop with a label and then use the label in **continue** statement.
- The label name is optional, and is usually only used when we wish to return to the outermost loop in a series of nested loops.

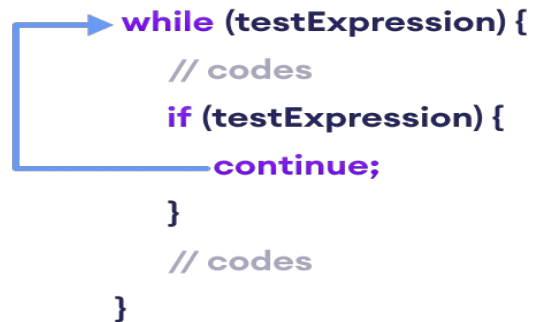
Syntax:

`continue; // the unlabeled form`

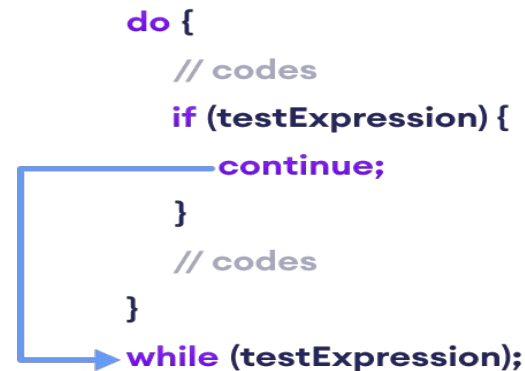
`continue <label>; // the labeled form`

jump statements- continue statement

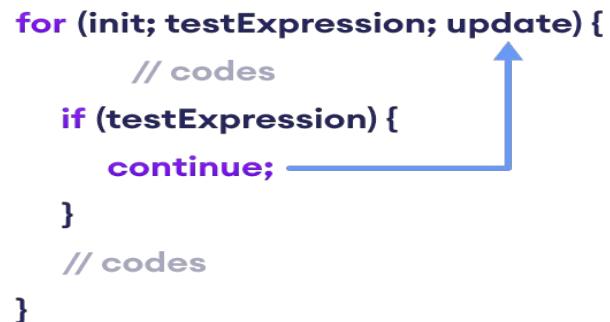
```
while (testExpression) {  
    // codes  
    if (testExpression) {  
        continue;  
    }  
    // codes  
}
```



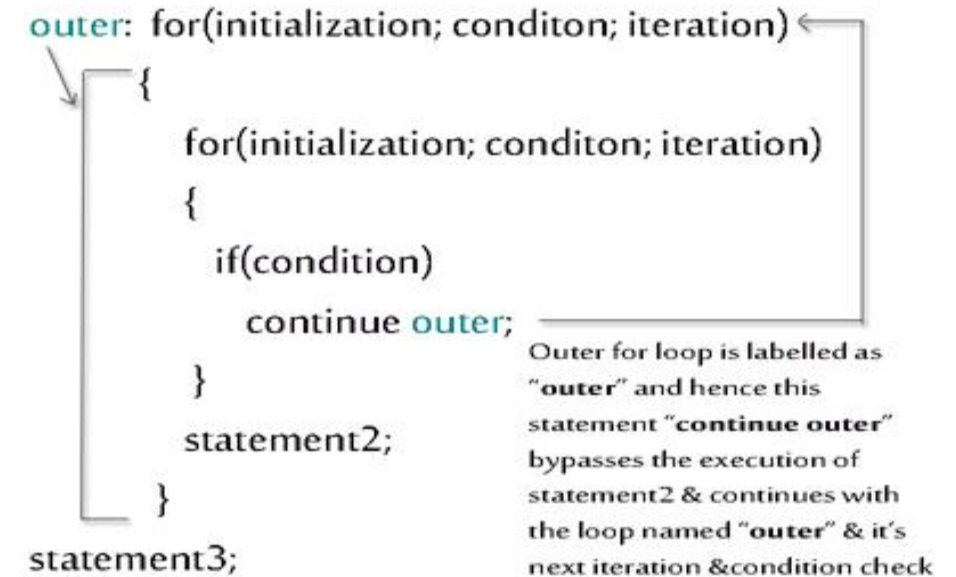
```
do {  
    // codes  
    if (testExpression) {  
        continue;  
    }  
    // codes  
} while (testExpression);
```



```
for (init; testExpression; update) {  
    // codes  
    if (testExpression) {  
        continue;  
    }  
    // codes  
}
```



```
outer: for(initialization; conditon; iteration) {  
    {  
        for(initialization; conditon; iteration)  
        {  
            if(condition)  
                continue outer;  
        }  
        statement2;  
    }  
    statement3;  
}
```



Outer for loop is labelled as "outer" and hence this statement "continue outer" bypasses the execution of statement2 & continues with the loop named "outer" & it's next iteration & condition check

labelled continue

jump statements- continue statement

```
public class Example
{
    public static void main(String[] args)
    {
        System.out.println("Numbers");
        for (int i = 1; i <= 10; i++)
        {
            if (i % 2 == 0)
                continue;
            System.out.println(i);
        }
    }
}
```

OUTPUT:

Numbers

1
3
5
7
9

jump statements- continue statement

```
public class Example
{
    public static void main(String[] args)
    {
        System.out.println("Numbers");
        for (int i = 0; i < 10; i++)
        {
            System.out.print( i + " ");
            if (i % 2 == 0)
                continue;
            System.out.println();
        }
    }
}
```

OUTPUT:

Numbers

0 1

2 3

4 5

6 7

9 10

jump statements- continue statement

```
class ContinueLabel
{
    public static void main(String args[])
    {
        outer: for (int i=0; i<5; i++)
        {
            for(int j=0; j<5; j++)
            {
                if(j > i)
                {
                    System.out.println();
                    continue outer;
                }
                System.out.print(" " + (i * j));
            }
        }
        System.out.println();
    }
}
```

OUTPUT:

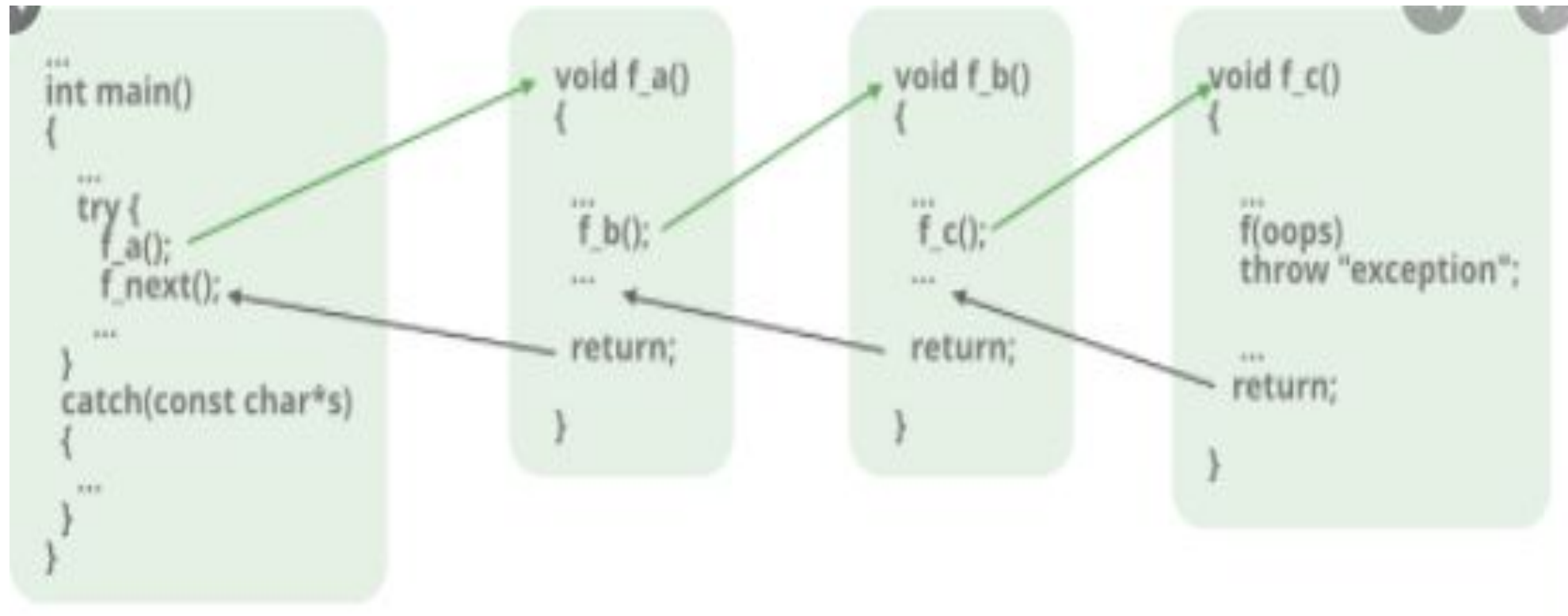
```
0
0 1
0 2 4
0 3 6 9
0 4 8 12 16
```

jump statements- return statement

The return statement

- A return statement causes the program control to transfer back to the caller of a method.
- At any time in a method/function the return statement can be used to cause execution to branch back to the caller of the method
- The return statement exits from the current method, and control flow returns to where the method was invoked.
- **Syntax:** The return statement has two forms:
 - One that returns a value : **return val;**
 - One that doesn't return a value : **return;**

jump statements- return statement



jump statements- continue statement

```
class Return
{
    public static void main(String args[])
    {
        boolean t = true;
        System.out.println("Before the Return");
        if(t)
            return; // return to the caller
        System.out.println("After the Return");
    }
}
```

OUTPUT:

Before the Return

“

Thank You!

”

ANY QUESTIONS?